

# “LifeLog – A Unified Personal Activity Intelligence Database”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

Database Systems Engineering and Distributed Backend Development · PBL 2026–27

**By**

**HAADI HASIL**

**2520030297**

**JAISWAL REDDY**

**2520030135**

**RAGHAVA AKKINEPALLY**

**2520030271**

Under the Esteemed Guidance of

**Dr. Y Prasanthi**

**Assistant Professor**

**Department of Computer Science and Engineering**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**Koneru Lakshmaiah Education Foundation (Deemed to be University)**  
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**Declaration**

The Project Report entitled “LifeLog – A Unified Personal Activity Intelligence Database” is a record of bonafide work of **HAADI HASIL – 2520030297, JAISWAL REDDY – 2520030135 and RAGHAVA AKKINEPALLY – 2520030271**, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to Koneru Lakshmaiah Education Foundation (K L Deemed to be University). The results embodied in this report have not been copied from any other department, university or institute.

**HAADI HASIL – 2520030297**  
**JAISWAL REDDY – 2520030135**  
**RAGHAVA AKKINEPALLY – 2520030271**

**Koneru Lakshmaiah Education Foundation (Deemed to be University)**  
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CERTIFICATE**

This is to certify that the project-based report entitled “LifeLog – A Unified Personal Activity Intelligence Database” is a bonafide work done and submitted by **HAADI HASIL – 2520030297, JAISWAL REDDY – 2520030135 and RAGHAVA AKKINEPALLY – 2520030271** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation (K L Deemed to be University), during the academic year **2026–2027**. The work was carried out under the guidance of Dr. Y Prasanthi, Assistant Professor, Department of Computer Science and Engineering.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to all those who assisted us, in one way or another, in the completion of this project.

Our greatest appreciation goes to our project guide, **Dr. Y Prasanthi**, Assistant Professor, Department of Computer Science and Engineering, for her tremendous support, encouragement and guidance throughout the project.

We express our gratitude to the **Head of the Department of Computer Science and Engineering** for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**HAADI HASIL – 2520030297**

**JAISWAL REDDY – 2520030135**

**RAGHAVA AKKINEPALLY – 2520030271**

## Abstract

Personal activity data is generated every day, but it is scattered across notebooks and unrelated applications that share no common data model. As a result, links between sleep, mood, study and spending are hard to analyse, summaries must be calculated by hand, and records may be lost, duplicated or inconsistent. **LifeLog – Unified Personal Activity Intelligence Database** addresses this problem by bringing these domains together in one normalized relational database.

LifeLog covers ten functional domains: daily activities, study sessions, expenses, food and nutrition, transport, screen time, mood, sleep, locations, and goals with their progress. The database, `lifelog_db`, contains fourteen related tables normalized up to Third Normal Form on the InnoDB engine. Its backbone is formed by `users` and `daily_logs`, from which the activity tables hang, while `activity_categories` and `locations` feed activities and goals feed `goal_progress`, which is updated by a trigger. Integrity is enforced with primary, foreign and composite keys, constraints, referential integrity and cascading rules.

Analysis is supported by SQL joins and aggregate queries, views, stored procedures, triggers, transactions and indexing. Operations are exposed through a Spring Boot 3.x REST API written in Java 17 and persisted through Spring Data JPA and Hibernate in MySQL 8+. A browser dashboard built with HTML, CSS, JavaScript, the Fetch API and Chart.js provides data-entry forms and visualization of trends. The project demonstrates the practical use of database concepts and turns fragmented personal records into one connected, queryable data system.

## Index

| S.No | Chapters                                            | Topics                                                                                                                                                                                                                                                                       | Page.no |
|------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|      | Declaration, Certificate, Acknowledgement, Abstract | Preliminary pages                                                                                                                                                                                                                                                            | 2-5     |
| 1    | <b>Introduction</b>                                 | 1.1 Background of the project<br>1.2 Problem statement; 1.3 Scope of the project<br>1.4 Objectives; 1.5 Importance of the application<br>1.6 Target users / audience; 1.7 Organization of the report                                                                         | 9-11    |
| 2    | <b>System Requirements</b>                          | 2.1 Hardware requirements<br>2.2 Software requirements<br>2.3 Development tools and frameworks                                                                                                                                                                               | 12-13   |
| 3    | <b>Technology Stack</b>                             | 3.1 Front-end; 3.2 Back-end; 3.3 Database<br>3.4 Version control; 3.5 APIs or external services                                                                                                                                                                              | 14-15   |
| 4    | <b>System Architecture</b>                          | 4.1 High-level architecture diagram<br>4.2 Description of each layer<br>4.3 Deployment architecture                                                                                                                                                                          | 16-18   |
| 5    | <b>Design</b>                                       | 5.1 Design overview; 5.2 ER diagram / database schema<br>5.3 Normalization<br>5.4 Keys, constraints and referential integrity<br>5.5 Database objects; 5.6 User interface design                                                                                             | 19-29   |
| 6    | <b>Implementation</b>                               | 6.1 Module-wise implementation; 6.2 Front-end logic<br>6.3 API integration; 6.4 Authentication & authorisation<br>6.5 SQL and DBMS implementation                                                                                                                            | 30-39   |
| 7    | <b>Features</b>                                     | 7.1 List of main features<br>7.2 How each feature works (user perspective + technical description)                                                                                                                                                                           | 40-41   |
| 8    | <b>Testing</b>                                      | 8.1 Postman testing (frontend/backend)<br>8.2 Integration testing<br>8.3 Tools used for testing<br>8.4 Test cases and results                                                                                                                                                | 42-46   |
| 9    | <b>Deployment</b>                                   | 9.1 Steps to deploy<br>9.2 Environment configuration<br>9.3 Hosting of frontend and backend                                                                                                                                                                                  | 47-48   |
| 10   | <b>Challenges &amp; Limitations</b>                 | 10.1 Issues faced during development<br>10.2 Solutions applied<br>10.3 Current limitations or known bugs                                                                                                                                                                     | 49-50   |
| 11   | <b>Future Enhancements</b>                          | 11.1 Planned features<br>11.2 Possible integrations or optimisations                                                                                                                                                                                                         | 51-52   |
| 12   | <b>Conclusion</b>                                   | 12.1 Summary of the project<br>12.2 What was achieved<br>12.3 Skills learned during development                                                                                                                                                                              | 53-54   |
| 13   | <b>References</b>                                   | Books, technology documentation, tutorials, APIs and tools, project source materials                                                                                                                                                                                         | 55      |
| 14   | <b>Appendices</b>                                   | Screenshots of the app; architecture and ER diagrams<br>Database schema; sample code snippets<br>Installation / setup instructions; user manual or guide<br>API documentation; testing evidence; repository<br>Geo-tag photos with guide; review forms with guide signatures | 56-60   |

## LIST OF FIGURES

| Figure      | Caption                                                                                          | Page |
|-------------|--------------------------------------------------------------------------------------------------|------|
| Figure 4.1  | System architecture of LifeLog (source: project review presentation)                             | 16   |
| Figure 5.1  | Simplified ER diagram of the fourteen tables of lifelog_db (source: project review presentation) | 19   |
| Figure 5.2  | Detailed ER diagram of lifelog_db with attributes, keys and cardinalities                        | 20   |
| Figure 6.1  | LifeLog dashboard                                                                                | 35   |
| Figure 6.2  | Data-entry interface                                                                             | 35   |
| Figure 6.3  | Database / analytics view                                                                        | 35   |
| Figure 8.1  | Postman request and response evidence                                                            | 43   |
| Figure 14.1 | Login / registration screen                                                                      | 56   |
| Figure 14.2 | Daily log and activities screen                                                                  | 56   |
| Figure 14.3 | Expenses screen                                                                                  | 56   |
| Figure 14.4 | Study, health and transport screens                                                              | 56   |
| Figure 14.5 | Screen time and goals screens                                                                    | 56   |
| Figure 14.6 | Analytics dashboard with Chart.js charts                                                         | 57   |
| Figure 14.7 | Full-page ER diagram of lifelog_db (larger version of Figure 5.2)                                | 57   |
| Figure 14.8 | Query output, trigger, view, procedure and transaction evidence                                  | 59   |

## LIST OF TABLES

| Table      | Caption                                                              | Page |
|------------|----------------------------------------------------------------------|------|
| Table 1.1  | Project objectives and where they are addressed                      | 10   |
| Table 2.1  | Recommended hardware configuration for development and demonstration | 12   |
| Table 2.2  | Software requirements                                                | 12   |
| Table 3.1  | Technology stack of LifeLog                                          | 14   |
| Table 4.1  | Responsibilities of the layers                                       | 17   |
| Table 5.1  | Relationships between the tables of lifelog_db                       | 20   |
| Table 5.2  | Summary of the tables of lifelog_db                                  | 21   |
| Table 5.3  | Anomalies of a single wide table and how LifeLog avoids them         | 26   |
| Table 5.4  | Functional dependencies reflected in the schema                      | 26   |
| Table 5.5  | Integrity mechanisms and what each protects                          | 28   |
| Table 5.6  | Database objects of the intelligence layer                           | 29   |
| Table 6.1  | Modules of LifeLog and the tables they use                           | 30   |
| Table 6.2  | Correspondence between HTTP methods, CRUD operations and SQL         | 36   |
| Table 6.3  | SQL concepts and their use in LifeLog                                | 37   |
| Table 7.1  | Main features of LifeLog                                             | 40   |
| Table 8.1  | Test areas and expected results defined in the project review        | 42   |
| Table 8.2  | API test plan for each module                                        | 42   |
| Table 8.3  | Integration test scenarios                                           | 43   |
| Table 8.4  | Tools and what they verify                                           | 44   |
| Table 8.5  | Functional and CRUD test cases                                       | 44   |
| Table 8.6  | Database test cases                                                  | 45   |
| Table 8.7  | Validation, negative and boundary test cases                         | 45   |
| Table 9.1  | Configuration items                                                  | 47   |
| Table 10.1 | Challenges and the solutions applied                                 | 49   |
| Table 10.2 | Current limitations                                                  | 50   |
| Table 11.1 | Future work and its dependence on the current design                 | 52   |
| Table 12.1 | Objectives achieved and the supporting evidence                      | 53   |
| Table 12.2 | Feasibility of the project                                           | 54   |
| Table 14.1 | Schema listing                                                       | 57   |
| Table 14.2 | REST endpoint listing                                                | 58   |

# CHAPTER -1 INTRODUCTION

## 1.1 Background of the Project

Every person generates a continuous stream of personal data. Over a single day a student attends classes and studies, spends money, eats several meals, travels between places, spends time on a phone or laptop, sleeps for some number of hours and experiences changes of mood, while also working towards longer-term goals. Each of these activities is small on its own, but taken together they describe how a person actually lives, works and spends time. Understanding that pattern is valuable, because habits that are invisible day by day often become visible only when the records are viewed side by side over weeks and months.

In practice, this information is rarely stored in one place. Expenses may be noted in a payment application, study hours in a notebook, sleep in a health application, screen time in a device setting and goals in a to-do list or only in memory. Each tool has its own structure, its own way of identifying a day and its own export format, and none of them is designed to be combined with the others. As a result, a question that is easy to ask, such as whether short sleep is followed by lower study time or by higher spending, is very hard to answer, because the data needed to answer it lives in unrelated systems that share no common data model.

This is precisely the kind of problem for which a relational database management system is well suited. A relational database stores data as related tables, enforces rules about what a valid record is, and lets related records be combined through joins and summarised through aggregate queries. **LifeLog – Unified Personal Activity Intelligence Database** applies these ideas to personal life data. It was developed by Haadi Hasil, Jaiswal Reddy and Raghava Akkinapally under the guidance of Dr. Y Prasanthi, Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, as a project for the course Database Systems Engineering and Distributed Backend Development (PBL 2026–27).

The core idea of LifeLog is expressed in the project tagline: turning everyday personal activity into meaningful, connected insight. All personal records are held in a single normalized relational database named `lifelog_db`, which contains fourteen related tables. The database is exposed to the user through a Spring Boot REST backend and a browser-based dashboard, so that a user can enter data through forms, retrieve it later and view summaries and trends in the form of charts. The project therefore combines database design, which is its central contribution, with a complete but compact full-stack application that makes the database usable.

## 1.2 Problem Statement

Personal activity data exists in silos. The project presentation identifies six concrete difficulties that follow from this situation, and each of them motivates a part of the LifeLog design.

First, data is scattered across notebooks and unrelated applications, so there is no single place where a person can see what they studied, spent, ate and did on a given day. Second, the different domains share no common data model: an expense record, a sleep record and a study record are not related to one another in any formal way, even though they all describe the same person on the same day. Third, as a direct consequence, the links between sleep, mood, study and spending are hard to analyse, because relating them requires manually copying numbers from one source to another. Fourth, trends and summaries such as weekly totals or averages must be calculated by hand. Fifth, retrieving historical records is slow and tedious, particularly when the records are on paper or spread across several applications. Sixth, data may be lost, duplicated or inconsistently recorded, since nothing enforces the correctness of what is entered.

The problem addressed by LifeLog is therefore not the absence of tracking tools, because many such tools exist, but the absence of a shared, validated and queryable data model that connects the different aspects of daily life. The project poses the following question: can the activities of a person across study, spending, food, transport, screen time, sleep, mood, locations and goals be stored in one normalized relational database in such a way that integrity is enforced by the database itself and cross-domain analysis becomes a matter of writing queries rather than copying numbers?

### 1.3 Scope of the Project

The scope of LifeLog covers a centralized personal activity system consisting of a relational database, a REST backend and a web dashboard. Ten functional domains are included: daily activities, study sessions, expenses, food and nutrition, transport, screen time, mood, sleep, locations, and goals with their progress. Every record belongs to an authenticated user, and the records of different domains are connected through a common relational model whose backbone is the user and the daily log.

Within this scope the project delivers the design of the fourteen-table schema in Third Normal Form, the use of primary, foreign and composite keys, referential integrity with cascading rules, database objects such as views, stored procedures, triggers and transactions, and SQL-based analytics through joins and aggregate queries. On the application side it delivers REST operations implemented with Spring Boot and Spring Data JPA/Hibernate and a browser dashboard built with HTML, CSS, JavaScript, the Fetch API and Chart.js for data entry, viewing and visualization.

Certain things are deliberately outside the scope of the current work. The project does not claim performance benchmarks, does not implement native mobile applications, and does not integrate with external fitness or banking services. Reminders, report export, cloud hosting with backups and two-factor authentication are identified as future scope and are described separately in Chapter 11 so that they are not confused with implemented functionality.

### 1.4 Objectives of the Project

The objectives of LifeLog, as stated in the project presentation, combine database engineering goals with application goals. They are listed here together with the way in which each is addressed in the report.

**Table 1.1:** *Project objectives and where they are addressed*

| No. | Objective                                                    | Addressed in                          |
|-----|--------------------------------------------------------------|---------------------------------------|
| 1   | Design a normalized relational database                      | Chapter 5 (Sections 5.2 and 5.3)      |
| 2   | Record personal activities reliably                          | Chapter 6 (Section 6.1) and Chapter 7 |
| 3   | Track sleep, mood and screen time                            | Sections 6.1.6 to 6.1.8               |
| 4   | Manage personal goals and progress                           | Section 6.1.10                        |
| 5   | Enforce integrity with keys, constraints and triggers        | Section 5.4 and Section 5.5           |
| 6   | Summarize data using views, procedures and aggregate queries | Section 5.5 and Section 6.5           |
| 7   | Expose operations through a Spring Boot REST API             | Sections 4.2, 6.3 and 6.4             |
| 8   | Visualize trends in an analytics dashboard                   | Sections 6.2 and 7.2                  |

The first, fifth and sixth objectives are specifically database objectives and are the main basis on which the project is evaluated as a DBMS project. The second, third and fourth describe the functional coverage of the system, and the seventh and eighth describe the application layers that make the database usable by an end user.

## 1.5 Importance of the Application

LifeLog is important for three reasons. The first is practical. A single place for personal records saves effort and removes the duplication that arises when the same day is described in several tools. It also makes retrieval simple, because a record is located by querying the database rather than by searching notebooks.

The second reason is analytical. Because all domains are stored in one relational model, the relationships between them can be examined directly. Joins can relate a sleep record to the study sessions of the same day, and aggregate queries can summarise spending or screen time over a period. These are the same operations that databases perform in commercial systems, applied here to a domain that every student understands.

The third reason is educational. The project is a concrete demonstration of the concepts taught in a database systems course: entity design, keys, normalization, referential integrity, views, stored procedures, triggers, transactions and indexing, together with their connection to a modern application stack. A complete worked example of this kind helps to show why these concepts exist and how they interact, which is often hard to see from isolated exercises.

## 1.6 Target Users / Audience

The primary users of LifeLog are students and individuals who want to understand their own habits. Students are an especially suitable audience because their day naturally combines study, expenses, food, transport, screen use, sleep, mood and goals, which are precisely the domains modelled by the system. Because every record is owned by an authenticated user, the same database can serve several users without mixing their data.

A secondary audience consists of database students, teachers and evaluators, who can study the schema, the integrity mechanisms and the analytical queries as a worked example. A tertiary audience consists of developers who want to see how a normalized relational design is connected to a Spring Boot REST service and a lightweight browser client.

## 1.7 Organization of the Report

The report follows the chapter structure prescribed for project documentation. Chapter 2 lists the hardware and software requirements. Chapter 3 describes the technology stack and Chapter 4 the system architecture. Chapter 5 presents the design of the database, including the schema, normalization and integrity mechanisms. Chapter 6 explains the implementation module by module, together with the front-end logic, API integration and authentication. Chapter 7 describes the features from a user and a technical perspective. Chapter 8 covers testing, Chapter 9 deployment, Chapter 10 challenges and limitations, Chapter 11 future enhancements and Chapter 12 the conclusion. References and appendices follow in Chapters 13 and 14.

*Where the project materials supplied for this report do not contain a particular item of evidence, such as column-level definitions, endpoint paths, screenshots or measured test results, the corresponding section carries an explicit placeholder instead of invented content.*

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

LifeLog is a lightweight application, and its hardware requirements are those of a typical student development laptop. The system has three runtime components that can run on one machine: the MySQL database server, the Spring Boot application, and a web browser that displays the dashboard. The project materials do not specify a measured hardware baseline, so the figures in Table 2.1 are recommended working values for development and demonstration rather than benchmarked minimums.

**Table 2.1:** *Recommended hardware configuration for development and demonstration*

| Component    | Recommended configuration                            | Reason                                                            |
|--------------|------------------------------------------------------|-------------------------------------------------------------------|
| Processor    | Dual-core or better, 64-bit                          | Runs the Java 17 runtime, MySQL server and browser together       |
| Memory (RAM) | 8 GB recommended (4 GB workable)                     | Spring Boot, MySQL and an IDE are memory-hungry when run together |
| Storage      | A few GB of free disk space                          | JDK, MySQL, Maven repository, project sources and database files  |
| Display      | Standard laptop or desktop display                   | The dashboard is a browser-based interface                        |
| Network      | Local machine; internet for downloading dependencies | Maven downloads libraries during the first build                  |

Because the database holds personal activity records in text and numeric form, storage growth is slow. Even a long period of daily use produces a small database compared with media-heavy applications, so storage is not a limiting factor for the scale of use considered in the project.

### 2.2 Software Requirements

The software requirements follow directly from the technology stack presented in the project review. The backend is written in Java 17 using Spring Boot 3.x and Spring Data JPA, the database is MySQL 8 or later with the InnoDB storage engine, and the front end is a browser application using HTML, CSS and JavaScript. The application can be developed and run on Windows or Linux.

**Table 2.2:** *Software requirements*

| Category          | Software                                         | Role in LifeLog                                                    |
|-------------------|--------------------------------------------------|--------------------------------------------------------------------|
| Operating system  | Windows or Linux                                 | Development and execution environment                              |
| Language runtime  | Java 17                                          | Language level used by the Spring Boot 3.x backend                 |
| Backend framework | Spring Boot 3.x with Spring Data JPA (Hibernate) | REST API and object-relational persistence                         |
| Database server   | MySQL 8 or later (InnoDB)                        | Stores the fourteen-table lifelog_db schema and enforces integrity |
| Build tool        | Maven                                            | Dependency management and project build                            |
| Browser           | A modern web browser                             | Displays the dashboard, forms and Chart.js visualizations          |
| Version control   | Git and GitHub                                   | Source management and collaboration                                |

InnoDB is mentioned explicitly in the project because it is the MySQL storage engine that supports foreign keys, cascading rules and transactions. Without it, the referential integrity and transaction behaviour described in Chapter 5 would not be enforced by the database.

## **2.3 Development Tools and Frameworks**

Development is carried out with a Java-capable editor or IDE, Maven for builds, and a MySQL client for inspecting the schema and running queries. Git is used for version control and GitHub is used to host the repository. Postman is the natural tool for exercising the REST API during development, and the browser developer tools are used to observe the Fetch API requests made by the dashboard. The project materials list Maven, Git, GitHub and a web browser as the development tools; the choice of a specific IDE and of Postman is left to the developer.

At the framework level, Spring Boot removes most of the configuration work needed to expose a REST service and to connect it to a database, Spring Data JPA together with Hibernate maps application objects to rows of the MySQL tables, and Chart.js turns the result sets returned by the API into charts. This division of work allows the team to concentrate on the design of the database, which is the principal subject of the project, instead of low-level plumbing.

## CHAPTER -3 TECHNOLOGY STACK

The LifeLog technology stack is organized in the same layers as the architecture described in Chapter 4: a browser front end, a Spring Boot REST backend, a persistence layer provided by Spring Data JPA and Hibernate, and a MySQL database. Supporting tools handle building and source control. Table 3.1 summarises the stack as presented in the project review, and the sections that follow explain the role of each part and the reason it was chosen.

**Table 3.1:** *Technology stack of LifeLog*

| Layer       | Technologies                                                            | Purpose                                          |
|-------------|-------------------------------------------------------------------------|--------------------------------------------------|
| Frontend    | HTML, CSS, JavaScript, Fetch API, Chart.js                              | Data-entry forms, dashboard views and charts     |
| Backend     | Java 17, Spring Boot 3.x, Spring Data JPA, REST APIs                    | Application operations exposed as REST services  |
| Persistence | Spring Data JPA / Hibernate                                             | Maps application objects to MySQL tables         |
| Database    | MySQL 8+, InnoDB, SQL, Views, Stored Procedures, Triggers, Transactions | Relational storage, integrity and analytics      |
| Development | Maven, Git, GitHub, Web Browser, Windows / Linux                        | Build, version control and execution environment |

### 3.1 Front-End (HTML, CSS, JavaScript, Chart.js)

The front end of LifeLog is a browser-based web dashboard built with plain HTML, CSS and JavaScript. HTML defines the structure of the pages and forms, CSS controls their presentation, and JavaScript supplies the behaviour: reading what the user has entered, sending it to the backend, and updating the page with the data that comes back. No heavyweight front-end framework is used, which keeps the client small and makes the interaction between the browser and the REST API easy to follow.

Communication with the backend uses the browser Fetch API. A request is sent from JavaScript to a REST endpoint, and the response is processed and rendered without reloading the whole page. Visualization is handled by Chart.js, a JavaScript charting library that draws charts from the numerical data returned by the backend. In LifeLog the charts are the visible output of the database's aggregate queries: a summary computed in SQL becomes a chart on the dashboard.

The choice of a plain HTML, CSS and JavaScript client fits the aim of the project. LifeLog is a database project, and the front end exists to make the database usable and to display its analytical results. A simple client, delivered as ordinary web pages, can be opened in any modern browser on Windows or Linux and keeps the feasibility of the project within a single semester.

### 3.2 Back-End (Java 17, Spring Boot 3.x, Spring Data JPA)

The backend is a Spring Boot 3.x application written in Java 17 that exposes the operations of LifeLog as REST APIs. Spring Boot is chosen because it provides an embedded web server, sensible defaults and automatic configuration, so that a REST service connected to a relational database can be created with comparatively little setup code. Maven manages the dependencies and the build.

Within the backend, the request path described in the project review passes through a controller, which receives the HTTP request, and then through the service and JPA layer, which carries out the operation against the database. Spring Data JPA, with Hibernate as the JPA implementation, is responsible for the translation between Java objects and rows of the MySQL tables. In this way the application code works with objects while the database continues to hold normalized relational data.

The use of a layered backend has a practical advantage for a DBMS project. The rules that belong to the database, such as key constraints, foreign keys and triggers, remain in the database where they are enforced for every client, and the backend adds only the application logic that should not live in SQL, such as request handling and the association of each request with the authenticated user.

### **3.3 Database (MySQL 8+ with InnoDB)**

MySQL 8 or later is the database management system. The schema, named `lifelog_db`, uses the InnoDB storage engine, which supports transactions, foreign keys with cascading actions and row-level locking. These capabilities are essential to the design of LifeLog because the integrity of the connected tables, the behaviour of the goal-progress trigger and the consistency of multi-step operations all depend on them.

The database contains fourteen related tables normalized to Third Normal Form. Besides tables, the database holds the objects that the project calls its database intelligence layer: views for reusable analytical queries, stored procedures for reusable operations, triggers for automatic actions, transactions for consistent multi-step operations, and indexes for faster lookups. These objects are described in Section 5.5, and the SQL language features that they rely on are discussed in Section 6.5.

MySQL was selected because it is open source, widely used in industry and in teaching, available on both Windows and Linux, and well supported by Spring Data JPA and Hibernate. Its open-source licence also supports the economic feasibility of the project.

### **3.4 Version Control (Git and GitHub)**

Source code is managed with Git and hosted on GitHub. Version control matters in a team project for three reasons. It allows the three team members to work in parallel on the database scripts, the backend and the dashboard without overwriting each other's work. It preserves the history of the schema, so that a change in a table definition can be traced and, if necessary, reverted. It also provides a single reference point from which the project can be built and run again on another machine. The actual repository address is to be recorded in Appendix J.

### **3.5 APIs or External Services**

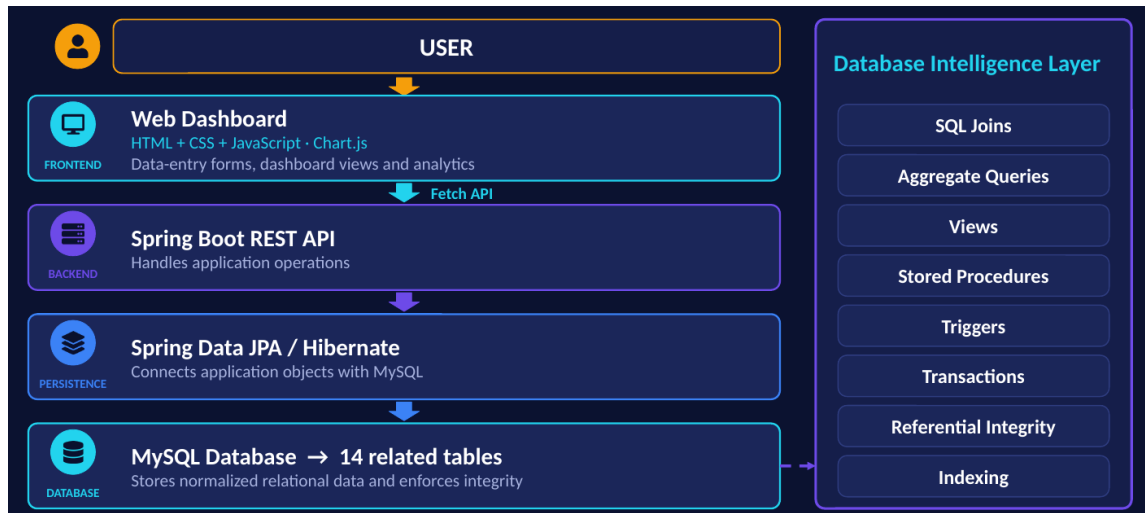
LifeLog exposes its own REST API, through which the dashboard performs all operations on the data. The API follows the usual REST conventions in which resources are addressed through URLs and manipulated with HTTP methods, and data is exchanged between the browser and the server in a structured form. The exact list of endpoints is to be taken from the final Spring Boot source code and is reserved for Appendix H.

The current reviewed scope uses no external third-party service. Integration with fitness or banking services is listed as future scope in Chapter 11 and is not part of the implemented system. The only external resources are therefore development dependencies, such as the libraries downloaded by Maven and the Chart.js library used by the dashboard.

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

LifeLog follows a layered architecture in which each layer has one responsibility and communicates only with the layer next to it. A user interacts with a web dashboard in the browser. The dashboard talks to a Spring Boot REST API through the Fetch API. The REST API uses Spring Data JPA and Hibernate to reach the MySQL database, which holds fourteen related tables and enforces integrity. Alongside this vertical path, the project identifies a database intelligence layer made up of the SQL and database features that give the system its analytical power: SQL joins, aggregate queries, views, stored procedures, triggers, transactions, referential integrity and indexing.



**Figure 4.1:** System architecture of LifeLog (source: project review presentation)

Figure 4.1 reproduces the architecture diagram of the project review. The left side shows the request path from the user down to the database, and the right side shows the database intelligence layer, which is attached to the database tier because all of its elements are features of the database itself.

The same architecture can be read as a chain of responsibilities, which is also the order in which a request travels: User, Frontend, Fetch API, Spring Boot REST API, Application and service layer, Spring Data JPA / Hibernate, MySQL. The project review summarises the runtime flow as User Input, REST Request, Controller, Service / JPA, MySQL, Response and finally Dashboard / Chart.

### 4.2 Description of Each Layer (Frontend, Backend, Database)

#### 4.2.1 Presentation layer (web dashboard)

The presentation layer is the web dashboard, built from HTML, CSS and JavaScript and enhanced with Chart.js. Its responsibility is to collect input through data-entry forms, to present stored records in dashboard views, and to turn the summaries returned by the backend into charts. It contains no business rules about the database and never accesses the database directly. All of its data operations are requests sent over the Fetch API.

#### 4.2.2 Backend layer (Spring Boot REST API)

The backend layer is the Spring Boot REST API, whose role in the architecture diagram is to handle application operations. It receives each HTTP request from the dashboard, identifies the operation requested, associates it with the authenticated user, passes it to the service layer and returns the result as a response. The backend is also the place where input is checked

before it reaches the database and where failures of the database layer, such as a rejected foreign key, are converted into responses that the dashboard can report to the user.

#### **4.2.3 Persistence layer (Spring Data JPA / Hibernate)**

The persistence layer connects application objects with MySQL. Hibernate implements the Java Persistence API and, together with Spring Data JPA, creates and runs the SQL needed to store and retrieve objects. Having this layer between the backend and the database means that the backend does not need to assemble SQL strings for routine create, read, update and delete operations, and that changes to the way an object is stored can be confined to one layer.

#### **4.2.4 Database layer (MySQL, fourteen related tables)**

The database layer is the MySQL database `lifelog_db`. It stores the normalized relational data and enforces integrity through primary keys, foreign keys, constraints and cascading rules. It also runs the logic that was deliberately placed inside the database: triggers that keep goal progress up to date, views and stored procedures that package analytical queries, and transactions that group steps into one consistent unit of work.

**Table 4.1: Responsibilities of the layers**

| Layer                       | Main responsibility                                         | What it must not do                                   |
|-----------------------------|-------------------------------------------------------------|-------------------------------------------------------|
| Frontend                    | Collect input, display records and charts                   | Access the database directly                          |
| Spring Boot REST API        | Receive requests, apply application logic, return responses | Duplicate integrity rules that belong to the database |
| Spring Data JPA / Hibernate | Map objects to table rows and run SQL                       | Contain presentation logic                            |
| MySQL                       | Store data and enforce integrity                            | Depend on a particular client                         |

#### **4.2.5 Request and response flow**

A write request, for example the recording of an expense, begins when the user submits a form. JavaScript sends the data to a REST endpoint through the Fetch API. The controller receives the request and hands it to the service layer, which uses JPA to issue an insert into the relevant table. MySQL checks the constraints and foreign keys, stores the row if it is valid and reports success or failure. The result travels back through the same layers as an HTTP response, and the dashboard updates its view. A read request follows the same path in the opposite direction for the data: MySQL returns rows, Hibernate turns them into objects, the controller serialises them in the response, and the dashboard renders them as lists or charts.

This separation of concerns provides three benefits. Errors can be traced layer by layer, from the browser request to the endpoint, then to the persistence layer and finally to the database. Each layer can be changed or tested largely independently of the others. And rules that must hold for all clients, such as referential integrity, live in the one component that every client must pass through.

### **4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)**

The deployment architecture of the current project is local. The MySQL server, the Spring Boot application and the browser run on a development or demonstration machine running Windows or Linux, and the dashboard communicates with the backend over the local network interface. This arrangement is sufficient for the semester project and keeps the economic cost at zero, because all of the technologies are open source.

The project materials do not describe a cloud environment or a continuous integration and delivery pipeline, and this report therefore does not claim one. A cloud-hosted database with backups is listed as future scope in Chapter 11. In such a

deployment the architecture of Figure 4.1 would remain unchanged and only the physical location of the layers would change: the MySQL layer would move to a managed cloud database, and the backend would be hosted on a server reachable from the browser. Section 9.3 discusses this transition.

## CHAPTER -5 DESIGN

### 5.1 Design Overview

LifeLog is a DBMS project, and its design is organised around the relational model. In the relational model data is held in relations, which in SQL are tables, and the meaning of the data is carried by keys and by the relationships between tables rather than by the physical arrangement of files. The design of LifeLog starts from three questions: which entities exist in personal activity data, how are they related, and how can the schema be arranged so that each fact is stored once and every relationship is guarded by the database.

The answer given by the project is a schema of fourteen related tables in the database `lifelog_db`, implemented on the InnoDB storage engine and normalized to Third Normal Form. The tables fall into four groups. The backbone consists of `users` and `daily_logs`. Two reference tables, `activity_categories` and `locations`, classify and place activities. Eight activity tables record the actual data of the days: `activities`, `expenses`, `food_logs`, `study_sessions`, `transport_logs`, `screen_time_logs`, `mood_logs` and `sleep_logs`. Finally, `goals` and `goal_progress` record intentions and their progress.

Two ideas shape the whole design. The first is the **backbone** of `users` and `daily_logs`. Every record is owned by a user and, for the activity tables, belongs to a logged day, so that data from every domain can be tied together by user and day. The second is that **integrity is enforced by the database**. Primary keys, foreign keys, composite keys where required, constraints, cascading rules and a trigger for goal progress are defined in the schema, so that the correctness of the data does not depend on the correctness of any particular client.

### 5.2 ER Diagram / Database Schema

The relationships between the fourteen tables are shown in the simplified ER diagram of the project review, reproduced in Figure 5.1. Boxes represent tables and arrows represent foreign-key relationships from a parent table to a child table. The dashed line labelled “`users → every log table`” indicates that each log table is owned by a user.

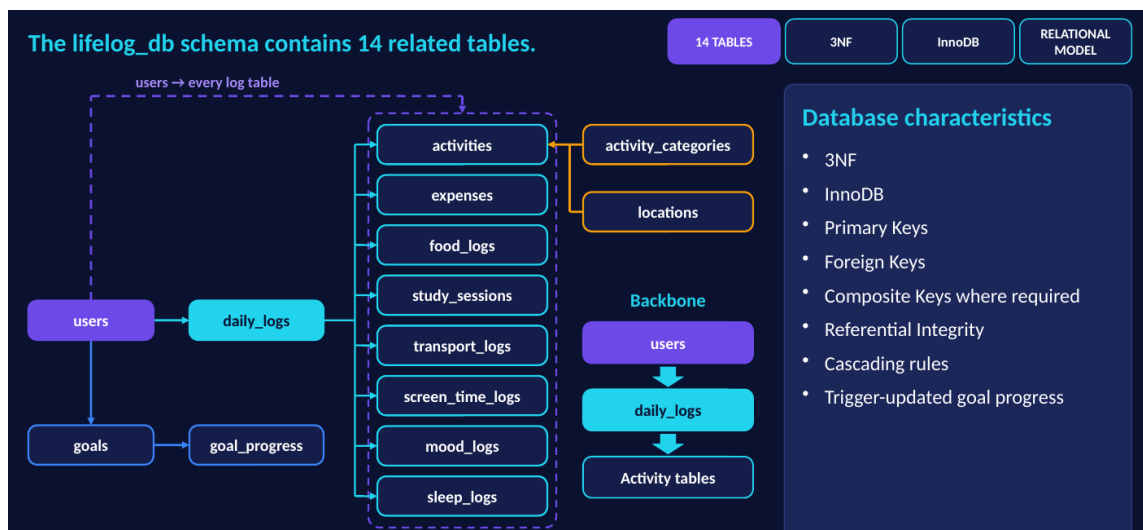


Figure 5.1: Simplified ER diagram of the fourteen tables of `lifelog_db` (source: project review presentation)

From the diagram, the structure of the schema can be read as follows. The `users` table leads to `daily_logs`, and `daily_logs` leads to the eight activity tables. The tables `activity_categories` and `locations` both lead into `activities`. The `users` table also leads to `goals`, and `goals` leads to `goal_progress`. The right-hand panel of the diagram repeats the backbone as `users`, then `daily_logs`, then the activity tables, and lists the database characteristics of the schema: Third Normal Form, InnoDB, primary keys, foreign keys, composite keys where required, referential integrity, cascading rules and trigger-updated goal progress.

**Table 5.1:** Relationships between the tables of lifelog\_db

| Parent table        | Child table(s)                                                                                           | Type        | Meaning                                           |
|---------------------|----------------------------------------------------------------------------------------------------------|-------------|---------------------------------------------------|
| users               | daily_logs                                                                                               | One-to-many | A user has many logged days                       |
| users               | goals                                                                                                    | One-to-many | A user defines many goals                         |
| users               | every log table                                                                                          | Ownership   | Each log record is owned by a user                |
| daily_logs          | activities, expenses, food_logs, study_sessions, transport_logs, screen_time_logs, mood_logs, sleep_logs | One-to-many | A logged day contains many records in each domain |
| activity_categories | activities                                                                                               | One-to-many | A category classifies many activities             |
| locations           | activities                                                                                               | One-to-many | A location is used by many activities             |
| goals               | goal_progress                                                                                            | One-to-many | A goal accumulates many progress records          |

Cardinality is stated at the level shown in the project diagram (parent to child, one-to-many). The exact foreign-key column names, nullability, any uniqueness rules such as one daily log per user per date, and the composite keys mentioned in the project review should be confirmed against the final DDL and added here.

**Figure 5.2:** Detailed ER diagram of lifelog\_db with attributes, keys and cardinalities

### 5.2.1 Entities and their roles

In ER terms each table is an entity set. users is a strong entity, because it does not depend on any other table for its existence. activity\_categories and locations are also strong entities that act as reference data. daily\_logs depends on users and the activity tables depend on daily\_logs, so they are dependent entities whose rows cannot exist without the corresponding parent row. goal\_progress is similarly dependent on goals. The relationships in the diagram are therefore identifying or ownership relationships: a child row is meaningful only in the context of its parent, and this is the reason for the cascading rules described in Section 5.4.

Attributes are the columns of each table. The project materials supplied for this report identify the tables, their keys at a conceptual level and their relationships, but they do not list column names and data types. Column-level definitions are therefore deliberately left as a placeholder in Table 5.2 and in Appendix D, and are not invented here.

### 5.2.2 Summary of the fourteen tables

**Table 5.2:** Summary of the tables of lifelog\_db

| No. | Table | Group    | Primary key                           | Columns and data types                              |
|-----|-------|----------|---------------------------------------|-----------------------------------------------------|
| 1   | users | Backbone | Yes (single or composite as required) | [To be updated with actual implementation evidence] |

| No. | Table               | Group     | Primary key                           | Columns and data types                              |
|-----|---------------------|-----------|---------------------------------------|-----------------------------------------------------|
| 2   | daily_logs          | Backbone  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 3   | activity_categories | Reference | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 4   | locations           | Reference | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 5   | activities          | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 6   | expenses            | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 7   | food_logs           | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 8   | study_sessions      | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 9   | transport_logs      | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 10  | screen_time_logs    | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 11  | mood_logs           | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 12  | sleep_logs          | Activity  | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 13  | goals               | Goals     | Yes (single or composite as required) | [To be updated with actual implementation evidence] |
| 14  | goal_progress       | Goals     | Yes (single or composite as required) | [To be updated with actual implementation evidence] |

### 5.2.3 Table-by-table description

This section explains each table in the same order. For each table it states the purpose, the keys and relationships, the reason why the table is separate, and an example use case.

#### 5.2.3.1 users

**Purpose.** The users table represents the people who own data in LifeLog. It is the root of the whole schema: every other record in the database is ultimately owned by a user, which is what allows LifeLog to serve several people from one database while keeping their data separate. It is also the table through which authentication is tied to the data.

**Keys and relationships.** Primary key identifies each user. It is the referenced (parent) table for daily\_logs and goals, and the project diagram shows an ownership link from users to every log table.

**Why it is a separate table.** User identity is a different entity from anything the user logs. Keeping it in its own table means that personal details are stored once, not repeated in every record, which is a direct requirement of Third Normal Form.

**Example use case.** A new student registers; the resulting row is the owner of every daily log, expense, sleep record and goal that the student later creates. **Attributes:** [To be updated with actual implementation evidence]

### 5.2.3.2 *daily\_logs*

**Purpose.** The `daily_logs` table is the second element of the backbone (users, then `daily_logs`, then activity tables). It represents one logged day of one user and gives every activity-specific table a common day-level anchor, so that records from different domains that belong to the same day can be related to each other.

**Keys and relationships.** Primary key identifies the logged day. It is a child of users and the parent of the eight activity tables shown in the project diagram: activities, expenses, `food_logs`, `study_sessions`, `transport_logs`, `screen_time_logs`, `mood_logs` and `sleep_logs`.

**Why it is a separate table.** The notion of a day is shared by all domains. If each domain table stored its own copy of the day description, those copies could disagree. A single `daily_logs` row removes that duplication and makes cross-domain joins on the day simple.

**Example use case.** To compare a night's sleep with the same day's study time and spending, a query joins `sleep_logs`, `study_sessions` and expenses through the same `daily_logs` row. **Attributes:** [To be updated with actual implementation evidence]

### 5.2.3.3 *activity\_categories*

**Purpose.** The `activity_categories` table is a reference (lookup) table that classifies activities. It lets the same category be reused by many activity records and ensures that the description of a category is maintained in exactly one place.

**Keys and relationships.** Primary key identifies the category. It is the parent in a one-to-many relationship with activities; the project diagram draws this relationship from `activity_categories` into activities.

**Why it is a separate table.** If the category description were stored in every activity row, renaming a category would require updating many rows (an update anomaly) and a typing difference would create two categories that mean the same thing. A separate table avoids both problems.

**Example use case.** Several activities recorded on different days all refer to one category row; a summary of activities per category is a GROUP BY over that relationship. **Attributes:** [To be updated with actual implementation evidence]

### 5.2.3.4 *locations*

**Purpose.** The `locations` table holds the places that can be associated with activities. It is the second reference table feeding the activities table and allows questions such as where a particular kind of activity usually takes place.

**Keys and relationships.** Primary key identifies the location. It is a parent of activities in a one-to-many relationship, as shown by the diagram.

**Why it is a separate table.** A place is an entity in its own right and is used by many activities. Storing it once avoids repeating place details and allows a place to be corrected in one row.

**Example use case.** A user logs several activities at the same place on different days; the place is stored once and referenced by each of those activities. **Attributes:** [To be updated with actual implementation evidence]

### 5.2.3.5 *activities*

**Purpose.** The `activities` table records the general activities that a user performs on a logged day. It is the table that receives both reference tables, `activity_categories` and `locations`, and therefore combines the classification of an activity with where it took place.

**Keys and relationships.** Primary key identifies the activity. Foreign keys connect it to a daily log and to the reference tables activity\_categories and locations, and ownership connects it to users.

**Why it is a separate table.** General activities are different in nature from specialised records such as expenses or sleep. Keeping them in their own table avoids a wide table with many columns that are empty for most rows.

**Example use case.** A user logs an evening walk as an activity of a given category at a given location on a given day.

**Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.6 expenses**

**Purpose.** The expenses table records money spent by a user on a logged day. It is the foundation of the spending analysis in LifeLog and the table most often summarised with SUM and GROUP BY queries.

**Keys and relationships.** Primary key identifies each expense. It is a child of daily\_logs and is owned by a user through the users link.

**Why it is a separate table.** Expenses have their own attributes and their own analytical queries, so they form a separate entity. Separation prevents repeating groups of expense columns inside the daily log.

**Example use case.** A student records a canteen payment; at the end of the week an aggregate query totals the expenses of the week and the dashboard plots the total. **Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.7 food\_logs**

**Purpose.** The food\_logs table records food and nutrition entries for a logged day. It supports the food and nutrition domain and allows eating patterns to be compared with other domains such as mood or sleep.

**Keys and relationships.** Primary key identifies each food entry. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** Several food entries may be recorded for one day. Placing them in a separate table with a foreign key to the daily log is the standard way to represent this one-to-many relationship without repeating columns.

**Example use case.** A user records three meals on one day; each meal is a separate row linked to the same daily log.

**Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.8 study\_sessions**

**Purpose.** The study\_sessions table records individual study sessions of a user. It supports the study domain and enables consistency analysis, such as how often and for how long study took place across days.

**Keys and relationships.** Primary key identifies each session. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** A day can contain many study sessions. Representing each session as a row, rather than storing a single daily total, retains detail and lets totals be derived by aggregation instead of being stored and maintained manually.

**Example use case.** A user records two sessions on one day; a query sums them to give the total study time for that day and compares it with the previous day. **Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.9 transport\_logs**

**Purpose.** The transport\_logs table records the travel of a user on a logged day. It supports the transport domain and makes it possible to summarise how a user travels over a period.

**Keys and relationships.** Primary key identifies each transport record. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** Travel is repeated within a day and has its own attributes. A separate table models the one-to-many relationship from daily log to journeys cleanly.

**Example use case.** A user logs a daily commute; the weekly summary of journeys is retrieved with a grouped query.

**Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.10 screen\_time\_logs**

**Purpose.** The screen\_time\_logs table records the time a user spends on screens. It supports the screen-time domain and provides the data needed to relate digital habits to sleep, mood and study.

**Keys and relationships.** Primary key identifies each screen-time record. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** Screen-time data is collected independently of other domains and analysed separately, so it is modelled as its own entity linked to the day.

**Example use case.** A user records screen time for a day; a query relates it to the same day's sleep record to see whether long screen time coincides with shorter sleep. **Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.11 mood\_logs**

**Purpose.** The mood\_logs table records how a user felt on a logged day. It is the subjective counterpart to the measurable domains and is central to the cross-domain questions that motivate LifeLog.

**Keys and relationships.** Primary key identifies each mood record. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** Mood is an observation about the day rather than a property of another entity. A separate table avoids adding sparse columns to other tables and allows several mood observations to be kept if needed.

**Example use case.** A user records a mood after an exam; later, a join with study\_sessions and sleep\_logs examines the conditions of low-mood days. **Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.12 sleep\_logs**

**Purpose.** The sleep\_logs table records the sleep of a user. It supports the sleep domain and is one of the most important tables for the health-related analysis of the system.

**Keys and relationships.** Primary key identifies each sleep record. It is a child of daily\_logs and is owned by a user.

**Why it is a separate table.** Sleep has its own attributes and is analysed on its own and in relation to other domains, so it is a separate entity linked to the day through a foreign key.

**Example use case.** A user records the night's sleep; an aggregate query computes the average sleep over a week and the dashboard charts the trend. **Attributes:** [To be updated with actual implementation evidence]

#### **5.2.3.13 goals**

**Purpose.** The goals table holds the personal goals defined by a user. It allows the system to track intentions as well as behaviour and is the parent of the goal-progress records.

**Keys and relationships.** Primary key identifies each goal. It is a child of users, as shown by the project diagram, and the parent of goal\_progress.

**Why it is a separate table.** A goal is a long-lived entity that persists while many progress updates accumulate against it. It therefore needs its own table rather than being merged with progress.

**Example use case.** A user defines a goal for regular study; the goal remains in the table while progress is recorded against it over many days. **Attributes:** [To be updated with actual implementation evidence]

#### 5.2.3.14 goal\_progress

**Purpose.** The goal\_progress table records the progress made towards a goal. The project review states that goal progress is updated by a trigger, which means that the table reflects activity data automatically and does not rely entirely on manual updates.

**Keys and relationships.** Primary key identifies each progress record. It is a child of goals in a one-to-many relationship, as shown by the diagram.

**Why it is a separate table.** Progress changes over time while the goal itself stays fixed. Separating the two avoids update anomalies on the goal and keeps the history of progress.

**Example use case.** When the underlying records change, the trigger updates the corresponding progress so that the dashboard shows current progress without the user editing it by hand. **Attributes:** [To be updated with actual implementation evidence]

### 5.3 Normalization

Normalization is the process of organising the columns and tables of a relational database so that redundancy is minimised and anomalies are avoided. It is guided by functional dependencies, which state that the value of one set of attributes determines the value of another. The project review states that LifeLog is normalized up to Third Normal Form (3NF). This section explains what each normal form requires and how the structure of LifeLog reflects it, using the table structure of the schema.

#### 5.3.1 Redundancy and anomalies

A design that places all personal data in a single wide table would store the details of the user and of the day repeatedly in every row. Such a design suffers from three well-known anomalies. An **insertion anomaly** appears when a fact cannot be recorded without recording an unrelated one, for example when a new category could not be created until some activity used it. An **update anomaly** appears when a repeated fact must be changed in many rows and a missed row leaves the database inconsistent. A **deletion anomaly** appears when deleting one record accidentally removes the only copy of another fact, for example when the last activity of a location is deleted and the location disappears with it.

**Table 5.3:** Anomalies of a single wide table and how LifeLog avoids them

| Anomaly    | Wide-table scenario                                                 | LifeLog design                                                                                         |
|------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Insertion  | A category or location cannot exist until an activity uses it       | activity_categories and locations are independent tables that exist before any activity refers to them |
| Update     | Changing a category description requires editing every activity row | The description is stored once in activity_categories and referenced by key                            |
| Deletion   | Deleting the last activity of a place removes the place             | Deleting an activity row does not remove its location or category row                                  |
| Redundancy | User and day details repeated in every domain record                | Stored once in users and daily_logs; domain tables hold a key                                          |

#### 5.3.2 First Normal Form (1NF)

A relation is in First Normal Form when every attribute holds an atomic value, there are no repeating groups, and each row can be identified by a key. LifeLog satisfies 1NF by representing repetition with rows rather than with columns. A day with

several expenses, meals or study sessions does not carry numbered columns such as expense1, expense2, expense3; instead each expense, meal or study session is its own row that points to the daily log through a foreign key. The same reasoning applies to every activity table. Each table also has a primary key, so each row is uniquely identifiable.

### 5.3.3 Second Normal Form (2NF)

A relation is in Second Normal Form when it is in 1NF and every non-key attribute depends on the whole of the primary key rather than on part of it. This condition matters only for tables with a composite key, which the project review lists among the key types where required. LifeLog keeps the attributes of each table about a single subject, so that a non-key attribute describes the entity identified by the whole key. Facts that would depend on only part of a composite key, such as the details of a user or of a category, are moved into their own tables, users and activity\_categories, and are referenced by foreign key.

### 5.3.4 Third Normal Form (3NF)

A relation is in Third Normal Form when it is in 2NF and no non-key attribute depends on another non-key attribute, that is, there are no transitive dependencies. Formally, for every functional dependency  $X \rightarrow A$  in the relation, either  $X$  is a superkey or  $A$  is a prime attribute. The split into reference tables is the clearest application of this rule in LifeLog. If an activity row stored the category description directly, then the description would depend on the category rather than on the activity key, giving the chain activity  $\rightarrow$  category  $\rightarrow$  description. Placing the category in activity\_categories and keeping only a foreign key in activities removes the transitive dependency. The same argument justifies locations as a separate table and the separation of goal\_progress from goals.

**Table 5.4:** Functional dependencies reflected in the schema

| Determinant                        | Determines                                                | Table that holds the dependency                                                                          |
|------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Primary key of users               | All descriptive attributes of the user                    | users                                                                                                    |
| Primary key of daily_logs          | The user and the day to which the log belongs             | daily_logs                                                                                               |
| Primary key of activity_categories | All descriptive attributes of the category                | activity_categories                                                                                      |
| Primary key of locations           | All descriptive attributes of the location                | locations                                                                                                |
| Primary key of an activity table   | The attributes of that single record and its foreign keys | activities, expenses, food_logs, study_sessions, transport_logs, screen_time_logs, mood_logs, sleep_logs |
| Primary key of goals               | The attributes of the goal and its owner                  | goals                                                                                                    |
| Primary key of goal_progress       | The progress attributes and the goal to which they belong | goal_progress                                                                                            |

In every row of Table 5.4 the determinant is the primary key of the table. When all functional dependencies have a key as determinant, the relation is in Boyce–Codd Normal Form as well as 3NF, but the project review claims only 3NF and this report follows that claim. The attribute-level functional dependencies, with the actual column names, should be added from the final schema.

Normalization has a cost, which is that queries across domains require joins. LifeLog accepts this cost deliberately because joins are the means through which cross-domain analysis is carried out, and because the indexing strategy and the views described in Section 5.5 reduce the effort of writing and executing them.

## 5.4 Keys, Constraints and Referential Integrity

### 5.4.1 Primary, foreign and composite keys

A **primary key** uniquely identifies each row of a table and cannot be null. Every table of lifelog\_db has a primary key. A **foreign key** is a column, or set of columns, in a child table that refers to the primary key of a parent table; it is the mechanism by which the relationships of Figure 5.1 are represented and enforced. A **composite key** is a key made of more than one column. The project review states that composite keys are used where required, that is, where a single column does not identify a row, typically in a table that resolves a relationship between two entities. The identification of the tables that use composite keys, and of their columns, is to be taken from the final DDL.

### 5.4.2 Constraints

Constraints are rules declared in the schema that a row must satisfy in order to be stored. They include NOT NULL for mandatory values, UNIQUE for values that must not repeat, CHECK for value conditions, DEFAULT for values assigned when none is given, and the key constraints already described. The testing plan of the project lists constraints as an area to verify with the expectation that invalid data is rejected. By placing these rules in the database, LifeLog guarantees that bad data is refused regardless of whether it comes from the dashboard, from a direct API call or from a SQL client. The specific constraints declared for each column are to be listed from the final DDL.

### 5.4.3 Referential integrity and cascading rules

Referential integrity requires that every foreign-key value either is null (where permitted) or matches an existing primary-key value in the parent table. In practice this prevents orphan records: an expense cannot refer to a daily log that does not exist, and goal progress cannot refer to a missing goal. The project review lists referential integrity and cascading rules as characteristics of the schema, and foreign keys as a test area in which invalid relationships are prevented.

Cascading rules define what the database does with child rows when a parent row is deleted or its key changes. With ON DELETE CASCADE the child rows are removed together with the parent; with RESTRICT or NO ACTION the deletion of a parent that still has children is refused; with SET NULL the foreign key in the children is cleared. For the dependent entities of LifeLog, cascading deletes are the natural expression of ownership, because log records have no meaning without the user and the day that own them. The action chosen for each foreign key is to be confirmed from the final DDL and recorded in Appendix D.

**Table 5.5:** *Integrity mechanisms and what each protects*

| Mechanism       | What it protects against                      | Where it applies in LifeLog                   |
|-----------------|-----------------------------------------------|-----------------------------------------------|
| Primary key     | Duplicate or unidentifiable rows              | Every table                                   |
| Foreign key     | Records that refer to non-existent parents    | Relationships shown in Figure 5.1             |
| Composite key   | Duplicate combinations in relationship tables | Where required (to be confirmed)              |
| Constraints     | Invalid or missing values                     | Columns of the activity, goal and user tables |
| Cascading rules | Orphaned child rows after parent deletion     | Parent–child links of the backbone and goals  |
| Trigger         | Stale goal progress                           | goal_progress                                 |
| Transactions    | Half-completed multi-step operations          | Multi-step operations                         |

## 5.5 Database Objects

Besides tables, the schema uses the database objects that the project groups under the database intelligence layer. These objects move repeated or critical logic into the database, where it can be shared by every client and tested independently of the application.

### 5.5.1 Views

A view is a stored SELECT statement that behaves like a virtual table. It does not hold data of its own; each time it is queried the underlying query is executed against the base tables. In LifeLog views package the analytical queries that are used repeatedly, such as summaries that join the user, the daily log and one or more activity tables. The benefit is that the complex join is written once, the dashboard can read a simple result, and a change in the underlying query is made in one place. The project testing plan expects that these reusable analytical queries work.

### 5.5.2 Stored procedures

A stored procedure is a named, precompiled block of SQL stored in the database and executed with a CALL statement. It can accept parameters and perform several statements in one call. In LifeLog stored procedures provide reusable operations, so that a multi-statement task can be run consistently by any client with a single call, and the logic remains close to the data.

### 5.5.3 Triggers

A trigger is a procedure that the database executes automatically when a specified event, such as an insert, update or delete on a table, occurs. The project review explicitly states that goal progress is updated by a trigger. This means that when relevant records change, the corresponding goal\_progress data is updated by the database itself, and neither the user nor the application has to remember to do it. The trigger thereby supports both the objective of enforcing integrity and the objective of managing goals and their progress. The event, the table and the exact update rule of the trigger are to be recorded from the final SQL scripts.

### 5.5.4 Transactions

A transaction is a group of statements treated as a single unit of work that has the ACID properties of atomicity, consistency, isolation and durability. Either all statements take effect (COMMIT) or none of them do (ROLLBACK). InnoDB provides this behaviour. The project review lists transactions as a means of keeping multi-step operations consistent, for example when several related rows must be created together and a failure part-way through must not leave a partial result.

### 5.5.5 Indexing

An index is an auxiliary data structure that allows rows to be found without scanning the whole table. Primary keys and foreign keys are normally indexed by InnoDB, and additional indexes can be created on columns that are searched or joined frequently. In LifeLog the columns involved in the joins between the backbone and the activity tables, and the columns used to filter by user and by date, are the natural candidates. Indexing is listed in the database intelligence layer of the architecture; the exact index definitions are to be taken from the final DDL, and no performance figures are claimed.

**Table 5.6:** Database objects of the intelligence layer

| Object            | Purpose                                   | Name(s) in the project                              |
|-------------------|-------------------------------------------|-----------------------------------------------------|
| Views             | Reusable analytical queries across tables | [To be updated with actual implementation evidence] |
| Stored procedures | Reusable multi-step operations            | [To be updated with actual implementation evidence] |

| Object       | Purpose                                                 | Name(s) in the project                              |
|--------------|---------------------------------------------------------|-----------------------------------------------------|
| Triggers     | Automatic update of goal progress and integrity actions | [To be updated with actual implementation evidence] |
| Transactions | Consistent execution of multi-step operations           | [To be updated with actual implementation evidence] |
| Indexes      | Faster lookups and joins                                | [To be updated with actual implementation evidence] |

## 5.6 User Interface Design

The user interface is designed as a browser dashboard that reflects the structure of the data. Data entry is organised by domain, so that a user records an expense, a study session, a meal, a journey, a screen-time entry, a mood, a sleep record or a goal through a form that corresponds to the matching table. Viewing is organised around the dashboard, which presents stored records and the summaries computed by the database in the form of charts drawn with Chart.js.

Two design decisions follow from the nature of the project. The first is that every screen operates on data belonging to the authenticated user and shows only that user's records. The second is that the user interface adds no rules about data validity that the database does not also enforce, so that invalid input is rejected in the same way whether it arrives through the dashboard or in any other way. Screenshots of the implemented interface are presented in Section 6.2 and in Appendix A.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Module-Wise Implementation

LifeLog is implemented as ten functional modules that correspond to the ten domains of the project: daily activities, study sessions, expenses, food and nutrition, transport, screen time, mood, sleep, locations, and goals with their progress. Each module is a vertical slice through the architecture of Chapter 4. It has a data-entry form and display area in the dashboard, a set of operations in the Spring Boot REST API, persistence through Spring Data JPA and Hibernate, and one or more tables in the database.

Because the modules follow the same pattern, the way in which they work can be described once. When a user submits a form, JavaScript sends the data to the REST API using the Fetch API. The controller receives the request, the service and JPA layer persist it, and MySQL validates the row against its keys and constraints before storing it. Reading follows the opposite direction, and the dashboard renders the returned records as lists or, for summaries, as Chart.js charts. The subsections below therefore concentrate on what is specific to each module: its purpose, the problem it addresses, the workflow, the data, the database representation, an example, its role and its analytical value.

**Table 6.1:** *Modules of LifeLog and the tables they use*

| No. | Module             | Main tables                                 |
|-----|--------------------|---------------------------------------------|
| 1   | Daily Activities   | daily_logs, activities, activity_categories |
| 2   | Study Sessions     | daily_logs, study_sessions                  |
| 3   | Expenses           | daily_logs, expenses                        |
| 4   | Food and Nutrition | daily_logs, food_logs                       |
| 5   | Transport          | daily_logs, transport_logs                  |
| 6   | Screen Time        | daily_logs, screen_time_logs                |
| 7   | Mood               | daily_logs, mood_logs                       |
| 8   | Sleep              | daily_logs, sleep_logs                      |
| 9   | Locations          | locations, activities                       |
| 10  | Goals and Progress | users, goals, goal_progress                 |

#### 6.1.1 Daily Activities

**Purpose and problem addressed.** The daily activities module is the general-purpose record of what a user does during a day. It uses the daily\_logs backbone and the activities table to hold activities that do not belong to one of the specialised domains. Without a shared record of the day, activities are remembered rather than recorded, and there is no common thing to which the other domains can be related.

**User workflow and data captured.** The user selects or creates the log for a day, chooses a category and a location, and saves the activity. The module captures the activity itself together with its classification by activity\_categories and its place by locations, for the logged day of the user.

**Database representation.** Rows of activities refer to daily\_logs, to activity\_categories and to locations, and are owned by the user. Reference rows are shared between many activities. **Example.** A user records a library visit as an activity of a study-related category at the campus location.

**Role in LifeLog and analytical value.** It anchors the day: the daily log created here is the row to which expenses, meals, sleep and every other record of the day attach. Counting and grouping activities by category or by location shows how time and places are distributed across days.

### 6.1.2 Study Sessions

**Purpose and problem addressed.** The study module records individual study sessions so that effort is documented as it happens instead of being estimated afterwards. Students rarely know how consistently they study, because study time is not recorded in the same place as the other facts of the day.

**User workflow and data captured.** The user opens the study form, records a session for the current day and saves it; sessions can be viewed and corrected later. The module captures what was studied and the session information needed to reason about study effort, as defined in the final schema of study\_sessions.

**Database representation.** Each session is a row of study\_sessions that refers to the daily log of the day and is owned by the user. A day with several sessions simply has several rows. **Example.** A student records a morning and an evening session on the same day, producing two rows under one daily log.

**Role in LifeLog and analytical value.** It represents the academic side of a student's life and links studying to the other domains through the shared day. Because sessions are stored individually, totals per day or per week are obtained by aggregation, and a join with sleep\_logs or mood\_logs can examine whether study quality follows sleep or mood.

### 6.1.3 Expenses

**Purpose and problem addressed.** The expense module records money spent by the user and is the financial dimension of LifeLog. Spending is usually tracked in payment applications that know nothing of sleep, mood or study, so its connection to behaviour is invisible.

**User workflow and data captured.** The user enters an expense on the expense form, saves it, and later views the list of expenses and the spending charts. The module captures the spending records of the user on a logged day, with the columns defined for expenses in the final schema.

**Database representation.** Rows of expenses refer to daily\_logs and are owned by the user. Each purchase is a row, so no repeating groups appear in the daily log. **Example.** A student records a canteen payment and a bus pass purchase on the same day as two expense rows.

**Role in LifeLog and analytical value.** It supplies the spending data that makes LifeLog useful for budgeting and for relating money to other habits. SUM and GROUP BY queries produce spending totals per day or week; ordering and filtering support the review of the largest or most recent expenses; a join with other domains links spending to the conditions of the day.

### 6.1.4 Food and Nutrition

**Purpose and problem addressed.** The food module records what the user eats, giving LifeLog a view of nutrition and eating patterns. Eating habits are seldom written down, so their possible influence on energy, mood and sleep cannot be examined.

**User workflow and data captured.** The user adds a food entry for a meal, saves it, and reviews the entries of the day. The module captures the food entries of a logged day, as defined for food\_logs in the final schema.

**Database representation.** Each meal is a row of food\_logs that refers to daily\_logs and is owned by the user, which allows any number of meals per day. **Example.** A user records breakfast, lunch and dinner as three rows under one daily log.

**Role in LifeLog and analytical value.** It completes the picture of physical well-being together with sleep and screen time. Counting entries per day and joining with mood\_logs or sleep\_logs supports questions about how eating relates to how the user feels and rests.

### **6.1.5 Transport**

**Purpose and problem addressed.** The transport module records the travel of the user, which is part of how time and money are spent during the day. Travel affects both time and cost, but it is not usually recorded alongside other activity data.

**User workflow and data captured.** The user records a journey on the transport form and saves it; journeys are listed on the dashboard. The module captures the journeys of the user for a logged day, with the columns defined for transport\_logs in the final schema.

**Database representation.** Each journey is a row of transport\_logs that refers to daily\_logs and is owned by the user.

**Example.** A student records a daily commute to college and a trip home as two rows of a day.

**Role in LifeLog and analytical value.** It adds the mobility dimension to the profile of the day and allows travel to be compared with spending and time use. Aggregate queries summarise travel over a week or month, and joins relate travel to expenses and to the time left for other activities.

### **6.1.6 Screen Time**

**Purpose and problem addressed.** The screen-time module records the time the user spends on screens, which is a major component of modern daily routine. Digital habits are among the most discussed influences on sleep, study and mood, yet their records are normally kept in a device setting that cannot be queried together with other data.

**User workflow and data captured.** The user records screen time for the day on the screen-time form and saves it; the dashboard shows the stored values. The module captures the screen-time records of the logged day, as defined for screen\_time\_logs in the final schema.

**Database representation.** Rows of screen\_time\_logs refer to daily\_logs and are owned by the user. **Example.** A user records screen time for a day on which the sleep record shows a short night.

**Role in LifeLog and analytical value.** It provides the quantitative evidence about digital habits that the project sets out to relate to other domains. Joins between screen\_time\_logs and sleep\_logs or study\_sessions on the shared day, and averages over a period, reveal patterns that no single application could show.

### **6.1.7 Mood**

**Purpose and problem addressed.** The mood module records how the user felt, adding a subjective dimension to the otherwise measurable data. Mood is a vital indicator of well-being but is rarely recorded, and when it is, it is separated from the circumstances that may explain it.

**User workflow and data captured.** The user records a mood entry for the day on the mood form and saves it; the entries can be reviewed over time. The module captures the mood records of the logged day, as defined for mood\_logs in the final schema.

**Database representation.** Each mood record is a row of mood\_logs that refers to daily\_logs and is owned by the user.

**Example.** A user records a low mood after an examination day and a better mood on the following day.

**Role in LifeLog and analytical value.** It supplies the outcome variable of many of the questions LifeLog can ask: how sleep, study, spending or screen time relate to mood. Trends of mood over time, and joins with the other domain tables on the same day, make the relationships between mood and daily behaviour visible.

#### **6.1.8 Sleep**

**Purpose and problem addressed.** The sleep module records the sleep of the user and is one of the principal health indicators of the system. Sleep influences study, mood and spending, but it is normally recorded in a health application whose data cannot be combined with study or expense records.

**User workflow and data captured.** The user records the sleep of the night on the sleep form and saves it; the dashboard charts the history. The module captures the sleep records of the logged day, as defined for sleep\_logs in the final schema.

**Database representation.** Each sleep record is a row of sleep\_logs that refers to daily\_logs and is owned by the user.

**Example.** A user records a short night before an examination and a longer night after it.

**Role in LifeLog and analytical value.** It is the most frequently joined table in the cross-domain analyses because many other quantities are expected to depend on it. Averages over a week and joins with study\_sessions, mood\_logs and screen\_time\_logs show how rest relates to the rest of the day.

#### **6.1.9 Locations**

**Purpose and problem addressed.** The locations module maintains the places that the user visits so that activities can be associated with where they happened. A place is part of the meaning of an activity, but without a shared record of places the same location may be described differently each time.

**User workflow and data captured.** The user adds a location once and then selects it when recording an activity. The module captures the place information defined for locations in the final schema.

**Database representation.** A row of locations is a reference row that can be used by many rows of activities through a foreign key. **Example.** A user adds the library and the hostel as locations and uses them repeatedly in daily activities.

**Role in LifeLog and analytical value.** It provides the spatial dimension of the data and illustrates the use of a reference table in a normalized design. Grouping activities by location shows where time is spent, and joins with activity\_categories show which kinds of activity take place where.

#### **6.1.10 Goals and Progress**

**Purpose and problem addressed.** The goals module lets a user define goals and follow progress towards them, turning the data of the other modules into a tool for self-improvement. Goals set in a notebook or in memory are easily forgotten, and nothing connects them to the records that show whether they are being met.

**User workflow and data captured.** The user creates a goal, and the progress is then shown on the dashboard as the recorded data changes. The module captures goals owned by the user and progress records attached to each goal, as defined for goals and goal\_progress in the final schema.

**Database representation.** A row of goals refers to users, and rows of goal\_progress refer to goals. The project review states that goal progress is updated by a trigger. **Example.** A user sets a goal for regular study, and the progress rows reflect the work recorded later without the user editing them.

**Role in LifeLog and analytical value.** It closes the loop of the system by comparing intentions with recorded behaviour. Progress values can be compared with goals, and the dashboard can chart progress over time; the trigger ensures that these comparisons reflect current data.

*The field-level content of each form and the exact REST endpoints of each module are not contained in the project materials supplied for this report and are listed as placeholders in Appendix H. The descriptions above stay at the level of the domains, tables and relationships that the project review defines.*

## 6.2 Front-End Logic (UI Rendering and Data Handling)

The dashboard is written in plain HTML, CSS and JavaScript, so the front-end logic consists of event handlers and functions that move data between forms, the Fetch API and the page. There is no framework-managed component state; the state of the interface is the data last returned by the server and the content of the page elements that display it.

### 6.2.1 Data entry and validation

Data entry is through HTML forms, one for each domain. A handler reads the values entered by the user, checks that required values are present and well formed, and sends the data to the backend. Client-side checks improve the experience by giving immediate feedback, but they are not the authority on validity: the backend and, finally, the database constraints decide whether a record is accepted, and a rejection is reported back to the user.

### 6.2.2 Communication with the API

Communication uses the Fetch API, which returns a promise for the HTTP response. The code sends the request, waits for the response, checks the status, reads the body and updates the page. The pattern below illustrates the structure only and is not a copy of the project source; the real endpoint paths and field names are in Appendix E and Appendix H.

```
// Illustrative pattern (not project code)
fetch('<endpoint-path>', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(<form-values>)
})
.then(response => { if (!response.ok) throw new Error(response.status); return
response.json(); })
.then(result => { /* update the page */ })
.catch(error => { /* show an error message */ });
```

### 6.2.3 Rendering records and charts

Returned records are rendered into tables or lists by creating page elements from the data. For analytics, the dashboard passes the numbers returned by the API to Chart.js, which draws the chart on a canvas element. The visible charts are thus the end of a chain that begins with an aggregate SQL query: the database computes the summary, the API delivers it and the browser draws it. The chart types used in the final dashboard are to be listed from the implemented screens.

### 6.2.4 Error handling and responsiveness

Failures can occur at several points, including a network failure, a request rejected by the backend, or a database constraint violation. The front end distinguishes between a successful and an unsuccessful response and informs the user instead of failing silently. The behaviour of the layout on small screens is to be confirmed from the implemented interface.

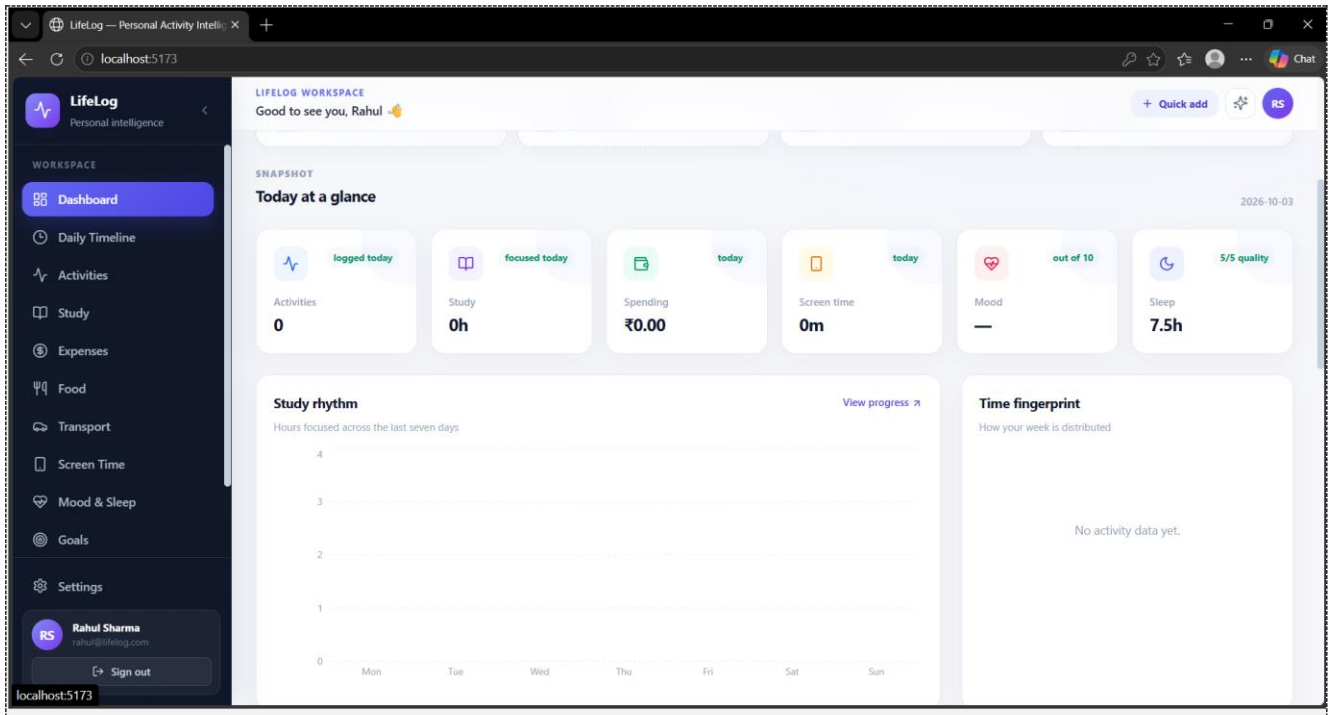


Figure 6.1: LifeLog dashboard

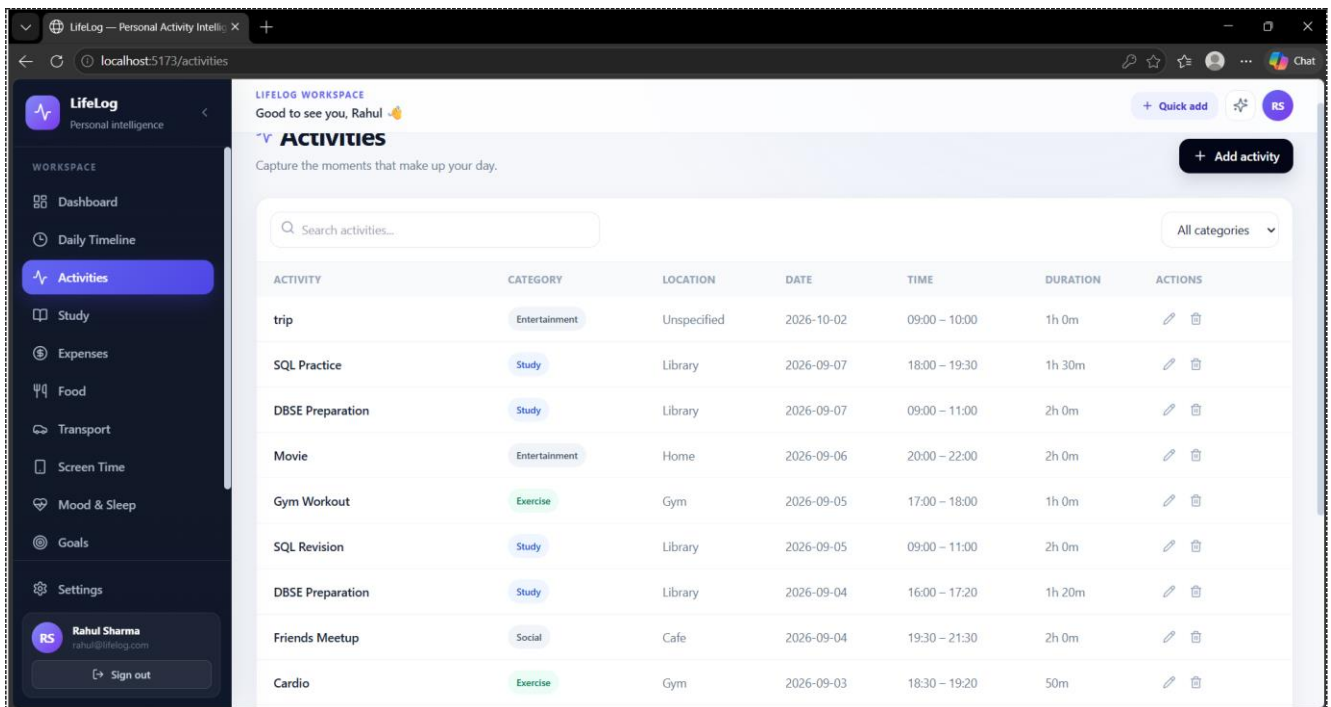
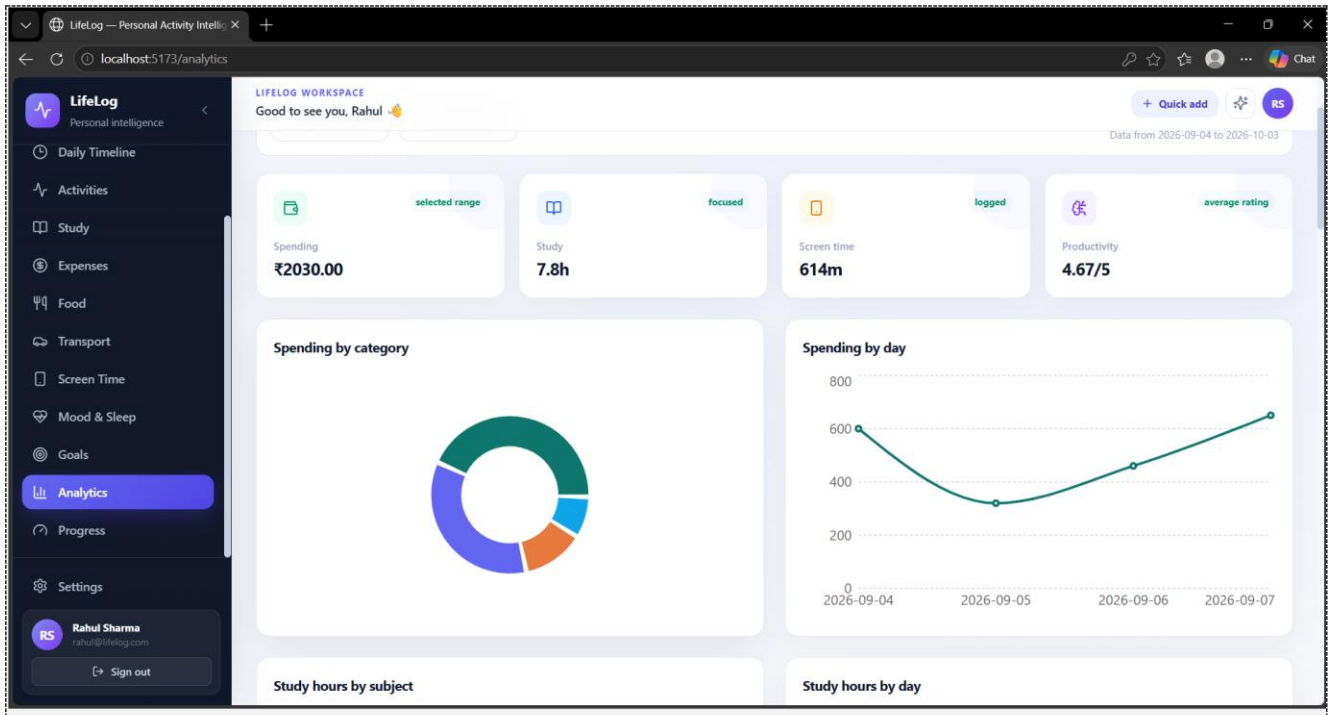


Figure 6.2: Data-entry interface



**Figure 6.3:** Database / analytics view

## 6.3 API Integration

The API is the contract between the dashboard and the backend. LifeLog uses REST, in which each kind of data is a resource identified by a URL and the operations on it are expressed with standard HTTP methods. The correspondence between HTTP methods, CRUD operations and SQL statements is the same for each module and is summarised in Table 6.2.

**Table 6.2:** Correspondence between HTTP methods, CRUD operations and SQL

| CRUD operation | Typical HTTP method | SQL statement executed through JPA |
|----------------|---------------------|------------------------------------|
| Create         | POST                | INSERT                             |
| Read           | GET                 | SELECT                             |
| Update         | PUT (or PATCH)      | UPDATE                             |
| Delete         | DELETE              | DELETE                             |

Table 6.2 states the standard REST convention. The actual methods and paths used in the LifeLog source code are to be recorded in Appendix H.

### 6.3.1 Create

Frontend → Fetch API → Spring Boot REST API → service and JPA/Hibernate → MySQL. The dashboard sends the new record, the backend creates the corresponding object and the persistence layer issues an INSERT. MySQL checks the primary key, foreign keys and constraints. If the record is valid it is stored, and a confirmation returns along the same path. If a foreign key points to a missing parent, or a constraint is violated, the insert fails and the error is reported instead of an inconsistent row being stored.

### 6.3.2 Read

MySQL → JPA/Hibernate → Spring Boot REST API → Fetch API → Frontend. A GET request is translated into a SELECT. The rows are converted into objects, serialised into the response and displayed by the dashboard. For analytical reads the SELECT can involve joins and aggregate functions, or the backend can read from a view that already contains them.

### 6.3.3 Update

An update identifies an existing record, changes it and persists the change with an UPDATE statement. Constraints are checked again, so an update cannot make a valid row invalid. If the updated data affects goal progress, the trigger of the database updates the progress as part of the same operation.

### 6.3.4 Delete

A delete removes a record with a DELETE statement. Referential integrity decides what happens to dependent rows: they are removed through cascading rules or the deletion is refused, depending on the rule defined for the foreign key. In either case the database is never left with child rows that refer to a parent that no longer exists.

### 6.3.5 Request lifecycle, validation and error handling

Every request follows the lifecycle User Input, REST Request, Controller, Service / JPA, MySQL, Response, Dashboard / Chart. Validation is applied at three levels. The browser checks basic input conditions. The backend checks that the request is acceptable and belongs to the authenticated user. The database applies keys and constraints, which are the final guarantee. Errors at each level are returned as HTTP error responses and displayed by the dashboard. The exact exception-handling classes and error formats are not included in the supplied materials and are to be taken from the source code.

## 6.4 Authentication and Authorisation

The project review states that personal records are stored as structured, validated data owned by an authenticated user. Authentication establishes who the user is; authorisation determines which data that user may see and change. In LifeLog these two ideas are tied to the data model itself. The users table is the root of ownership, the daily\_logs and goals tables refer to it, and the project diagram shows that every log table is owned by a user.

The practical consequence is per-user isolation. Every read or write that the backend performs is associated with the authenticated user, so that a user's dashboard shows only that user's records and one user cannot alter another user's data. Because ownership is represented by foreign keys, isolation rests on the same relational structure that provides integrity. The mechanism by which a user logs in and the way in which credentials are stored are not detailed in the supplied materials and are therefore marked as follows.

*[To be updated with actual implementation evidence] Authentication mechanism, session or token handling, and credential storage method. Two-factor authentication is listed in Chapter 11 as future scope and is not part of the implemented system.*

## 6.5 SQL and DBMS Implementation

This section links each SQL feature to its use in LifeLog. The project review names SQL joins, aggregate queries, views, stored procedures, triggers, transactions, referential integrity and indexing as the elements of the database intelligence layer. Other statements, such as CREATE, INSERT, UPDATE and DELETE, are the ordinary operations that any relational schema uses. No query results are shown, because the supplied materials contain no executed output. Where this report gives SQL, it is a pattern with angle-bracket placeholders that must be replaced by the real names from the project scripts in Appendix D and Appendix E.

**Table 6.3:** SQL concepts and their use in LifeLog

| Concept                           | Explanation                                                               | Use in LifeLog                                           |
|-----------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------|
| CREATE DATABASE /<br>CREATE TABLE | Define the database and the structure, keys and constraints of each table | Creation of lifelog_db and its fourteen tables on InnoDB |
| INSERT                            | Add rows                                                                  | Creation of every kind of log record                     |

| Concept                               | Explanation                                               | Use in LifeLog                                             |
|---------------------------------------|-----------------------------------------------------------|------------------------------------------------------------|
| SELECT, WHERE, ORDER BY               | Retrieve, filter and sort rows                            | Listing a user's records by day or in order                |
| UPDATE / DELETE                       | Change or remove rows                                     | Editing and removing records, with integrity rules applied |
| Aggregate functions, GROUP BY, HAVING | Summarise groups of rows; HAVING filters groups           | Totals and averages per day, week or category              |
| INNER JOIN / LEFT JOIN                | Combine related rows; LEFT JOIN keeps unmatched left rows | Relating backbone and activity tables across domains       |
| Subqueries                            | A query nested in another query                           | Possible in analytical queries (to be confirmed)           |
| Views                                 | Stored queries behaving as virtual tables                 | Reusable analytical queries                                |
| Stored procedures and functions       | Stored, parameterised routines                            | Reusable operations                                        |
| Triggers                              | Automatic actions on data events                          | Automatic update of goal progress                          |
| Transactions: COMMIT, ROLLBACK        | Atomic units of work                                      | Consistent multi-step operations                           |
| Constraints                           | Declarative rules for valid data                          | Rejection of invalid data                                  |

### 6.5.1 Joins

Joins are the operation that turns a normalized schema into an analytical tool. An INNER JOIN returns only rows that have a match on both sides, which is appropriate when a user and the logged days are to be listed together. A LEFT JOIN returns every row of the left table, with empty values where nothing matches, which is the right choice for finding days that have no record in a given domain. The pattern below shows the structure of a cross-domain query that relates the sleep and study records of the same day.

```
-- Pattern only: replace <...> with the real names from the project scripts
SELECT d.<day_column>, s.<sleep_value>, st.<study_value>
FROM daily_logs d
INNER JOIN sleep_logs s ON s.<daily_log_fk> = d.<pk>
LEFT JOIN study_sessions st ON st.<daily_log_fk> = d.<pk>
WHERE d.<user_fk> = <user_id>
ORDER BY d.<day_column>;
```

### 6.5.2 Aggregation

Aggregate functions such as COUNT, SUM, AVG, MIN and MAX reduce many rows to one value, and GROUP BY applies them per group. HAVING filters groups after aggregation, whereas WHERE filters rows before it. In LifeLog these are the queries that produce the numbers drawn on the dashboard, for example a total per period or an average per day.

```
-- Pattern only
SELECT <group_column>, SUM(<numeric_column>) AS total
FROM <activity_table>
WHERE <user_fk> = <user_id>
GROUP BY <group_column>
HAVING SUM(<numeric_column>) > <threshold>;
```

### 6.5.3 Views, procedures and triggers

A view is defined once with CREATE VIEW and queried like a table, a stored procedure is defined with CREATE PROCEDURE and executed with CALL, and a trigger is defined with CREATE TRIGGER together with the event (insert, update or delete) and timing (before or after) that fire it. The goal-progress trigger of LifeLog is the project's example of the last kind. The definitions below show the syntax only; the project's own definitions belong in Appendix D.

```
-- Syntax patterns only
CREATE VIEW <view_name> AS SELECT ... ;
CREATE PROCEDURE <procedure_name>(IN <param> <type>) BEGIN ... END;
CREATE TRIGGER <trigger_name> AFTER INSERT ON <table>
  FOR EACH ROW BEGIN ... END;
```

#### **6.5.4 Transactions**

A transaction groups statements so that they succeed or fail together. It begins with `START TRANSACTION`, ends with `COMMIT` when every statement has succeeded, and is undone with `ROLLBACK` when any statement fails. The pattern is relevant whenever one user action must change more than one row.

```
-- Pattern only
START TRANSACTION;
  INSERT INTO <table_a> ... ;
  INSERT INTO <table_b> ... ;
COMMIT;  -- or ROLLBACK; if any step fails
```

## CHAPTER -7 FEATURES

### 7.1 List of Main Features

The main features of LifeLog follow from its objectives and from the results listed in the project review. Table 7.1 lists them together with a short description. Each feature is a capability of the implemented system, and none of the features listed under future scope in Chapter 11 appears in this table.

**Table 7.1:** *Main features of LifeLog*

| No. | Feature                             | Description                                                                                                                         |
|-----|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Centralized personal data           | All activity data is held in one relational system instead of separate notebooks and applications.                                  |
| 2   | Structured recording of ten domains | Daily activities, study, expenses, food, transport, screen time, mood, sleep, locations and goals are stored as structured records. |
| 3   | Cross-domain analysis               | Joins and aggregation support analysis that relates one domain to another.                                                          |
| 4   | Reusable analytical operations      | Views and stored procedures give reusable operations for repeated analysis.                                                         |
| 5   | Data integrity                      | Triggers and constraints support the integrity of the data, and invalid data and invalid relationships are rejected.                |
| 6   | Automatic goal progress             | Progress towards goals is updated automatically.                                                                                    |
| 7   | Consistent multi-step operations    | Transactions keep multi-step operations consistent.                                                                                 |
| 8   | REST interface                      | The REST architecture gives the application a structured interface.                                                                 |
| 9   | Dashboard visualization             | Trends are visualized in a dashboard.                                                                                               |
| 10  | Per-user ownership                  | Records are owned by an authenticated user.                                                                                         |

### 7.2 How Each Feature Works (User Perspective and Technical Description)

This section explains each feature twice: first from the point of view of the user, who only sees the dashboard, and then from the technical point of view, which describes what happens in the layers beneath it.

#### 7.2.1 *Centralized personal data*

**User perspective.** The user records everything in one place, and the day is the same day in every domain. **Technical description.** A single database, `lifelog_db`, holds fourteen related tables. The users and `daily_logs` backbone ties every record to a person and a day.

#### 7.2.2 *Structured recording of ten domains*

**User perspective.** The user chooses the form that matches what is to be recorded and saves it. **Technical description.** Each domain has its own table with a foreign key to the relevant parent, which gives the activity tables a uniform structure without mixing their attributes.

#### 7.2.3 *Cross-domain analysis*

**User perspective.** The user sees summaries and trends that combine, for example, sleep and study. **Technical description.** SQL joins on the shared user and day, together with aggregate functions, compute the figures that the dashboard displays.

#### 7.2.4 *Reusable analytical operations*

**User perspective.** The user receives consistent summaries every time they are requested. **Technical description.** The complex query is defined once in the database as a view or procedure and is called by the backend.

### **7.2.5 Data integrity**

**User perspective.** The user cannot save a record that violates a rule or refers to something that does not exist. **Technical description.** Primary keys, foreign keys, constraints and cascading rules are enforced by InnoDB for every client.

### **7.2.6 Automatic goal progress**

**User perspective.** The user sets a goal and sees the progress change as records are logged, without updating progress by hand. **Technical description.** A database trigger updates goal\_progress when the relevant records change.

### **7.2.7 Consistent multi-step operations**

**User perspective.** The user never sees a half-saved result. **Technical description.** Related statements run inside a transaction and are committed together, or rolled back if any step fails.

### **7.2.8 REST interface**

**User perspective.** The user is not aware of the API; the dashboard simply works. **Technical description.** Spring Boot controllers expose operations that the dashboard calls with the Fetch API; Spring Data JPA persists the changes.

### **7.2.9 Dashboard visualization**

**User perspective.** The user sees charts of the recorded data instead of tables of numbers. **Technical description.** Chart.js draws the data returned by the REST API, which in turn comes from the aggregate queries.

### **7.2.10 Per-user ownership**

**User perspective.** The user sees only their own records. **Technical description.** Foreign keys to users and the authenticated-user association in the backend scope every query to the owner.

Taken together, the features show the central idea of the project. The user-visible behaviour is simple, and the guarantees behind it, such as consistency, isolation of users and automatic updates, are provided by relational mechanisms of the database rather than by repeated application code.

## CHAPTER -8 TESTING

Testing in LifeLog has two purposes. The first is to verify that the application behaves as intended, from the form in the browser to the row in the database. The second is to verify that the database enforces its own rules. The project review defines eight test areas and the result each should demonstrate, summarised in Table 8.1. This chapter expands those areas into test cases. Because the materials supplied for this report do not contain recorded execution results, the Actual Result and Status columns are marked as placeholders and must be filled in after the final execution. No result is claimed here.

**Table 8.1:** *Test areas and expected results defined in the project review*

| Test area       | Expected result                                        |
|-----------------|--------------------------------------------------------|
| CRUD operations | Records can be created, retrieved, updated and deleted |
| Foreign keys    | Invalid relationships are prevented                    |
| Constraints     | Invalid data is rejected                               |
| Joins           | Related records can be retrieved                       |
| Aggregations    | Summaries can be generated                             |
| Views           | Reusable analytical queries work                       |
| Triggers        | Automated integrity and progress actions execute       |
| Transactions    | Multi-step operations maintain consistency             |

### 8.1 Postman Testing (Frontend / Backend)

Postman is an application for sending HTTP requests to an API and inspecting the responses. For a REST backend such as LifeLog it allows each endpoint to be tested without the dashboard, which separates faults of the backend from faults of the front end. A request is prepared with a method, a URL and, for create and update operations, a body; the response is checked for the expected status code and content. Sending the same request through the dashboard and through Postman, and comparing the result, is a quick way to establish where a failure arises.

For each module the API test plan covers the four CRUD operations, plus the cases in which the backend or the database must refuse a request, such as a record that refers to a missing parent or lacks a required value. The endpoint paths are not contained in the supplied materials, so the endpoint column of Table 8.2 is a placeholder.

**Table 8.2:** *API test plan for each module*

| Module             | Create  | Read    | Update  | Delete  | Endpoint                                            |
|--------------------|---------|---------|---------|---------|-----------------------------------------------------|
| Daily activities   | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Study sessions     | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Expenses           | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Food and nutrition | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Transport          | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Screen time        | To test | To test | To test | To test | [To be updated with actual implementation evidence] |

| Module             | Create  | Read    | Update  | Delete  | Endpoint                                            |
|--------------------|---------|---------|---------|---------|-----------------------------------------------------|
| Mood               | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Sleep              | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Locations          | To test | To test | To test | To test | [To be updated with actual implementation evidence] |
| Goals and progress | To test | To test | To test | To test | [To be updated with actual implementation evidence] |

## 8.2 Integration Testing

Integration testing verifies that the layers work together. A working database does not guarantee a working application, because a fault may lie in the form, in the request, in the mapping performed by JPA or in the database rule. The integration tests of LifeLog therefore follow a complete path: user input in the dashboard, the Fetch request, the controller, the service and JPA layer, the MySQL operation and the response that returns to the dashboard.

Three kinds of path are of particular interest. In a successful write path the record entered in the browser must appear in the corresponding table and in the next read. In a rejected write path the record must not appear in the table and the user must see an error. In an analytical path the figures shown in a chart must agree with the result of the same query executed directly in a MySQL client. The last comparison is a particularly valuable check because it confirms that the dashboard presents what the database computes.

**Table 8.3:** *Integration test scenarios*

| ID    | Scenario                                  | Path covered                                  | Expected result                            | Actual Result                        | Status                               |
|-------|-------------------------------------------|-----------------------------------------------|--------------------------------------------|--------------------------------------|--------------------------------------|
| IT-01 | Create a record from the dashboard        | Form, Fetch, controller, JPA, MySQL, response | Record stored and visible in the next read | [To be filled after final execution] | [To be filled after final execution] |
| IT-02 | Submit a record with an invalid reference | Form, Fetch, controller, JPA, MySQL           | Rejected by foreign key; error shown       | [To be filled after final execution] | [To be filled after final execution] |
| IT-03 | Compare a chart with its SQL query        | MySQL, JPA, controller, Chart.js              | Chart values equal query result            | [To be filled after final execution] | [To be filled after final execution] |
| IT-04 | Log a record that affects a goal          | Form, API, MySQL trigger                      | Goal progress updated automatically        | [To be filled after final execution] | [To be filled after final execution] |
| IT-05 | Multi-step operation fails part-way       | API, transaction, MySQL                       | Whole operation rolled back                | [To be filled after final execution] | [To be filled after final execution] |

## 8.3 Tools Used for Testing

The tools used for testing are those that belong to the technology stack. Postman is used for API requests, the browser developer tools to observe Fetch requests, responses and console errors, a MySQL client to inspect tables and execute queries directly, and the Spring Boot application logs to follow what the backend and Hibernate execute. The report does not claim an automated unit-test suite or a continuous integration pipeline, since the supplied materials mention neither.

**Table 8.4:** *Tools and what they verify*

| Tool    | What it verifies                                         |
|---------|----------------------------------------------------------|
| Postman | Behaviour of REST endpoints independent of the dashboard |

| Tool                    | What it verifies                                         |
|-------------------------|----------------------------------------------------------|
| Browser developer tools | Fetch requests, responses and front-end errors           |
| MySQL client            | Stored data, constraints, views, procedures and triggers |
| Application logs        | Backend processing and the SQL issued by JPA             |

## 8.4 Test Cases and Results

The test cases below are grouped by type. Each lists an identifier, the module, the scenario, the input, the expected result, the actual result and the status. The expected results are derived from the behaviour defined in the project review and from the principles of the relational model. The actual results and statuses remain placeholders until the tests are executed on the final system.

### 8.4.1 Functional and CRUD testing

**Table 8.5:** Functional and CRUD test cases

| Test Case ID | Module         | Scenario                            | Input                                   | Expected Result                        | Actual Result                        | Status                               |
|--------------|----------------|-------------------------------------|-----------------------------------------|----------------------------------------|--------------------------------------|--------------------------------------|
| TC-01        | Expenses       | Create a valid expense              | Valid expense for an existing daily log | Record stored; success response        | [To be filled after final execution] | [To be filled after final execution] |
| TC-02        | Study sessions | Read the list of sessions of a user | Request for the authenticated user      | Only that user's sessions returned     | [To be filled after final execution] | [To be filled after final execution] |
| TC-03        | Sleep          | Update an existing sleep record     | Changed value for an existing record    | Record updated; new value on next read | [To be filled after final execution] | [To be filled after final execution] |
| TC-04        | Mood           | Delete a mood record                | Existing record identifier              | Record removed; no longer returned     | [To be filled after final execution] | [To be filled after final execution] |
| TC-05        | Goals          | Create a goal                       | Valid goal for the authenticated user   | Goal stored and linked to the user     | [To be filled after final execution] | [To be filled after final execution] |

### 8.4.2 Database testing

**Table 8.6:** Database test cases

| Test Case ID | Module       | Scenario                             | Input                                            | Expected Result                     | Actual Result                        | Status                               |
|--------------|--------------|--------------------------------------|--------------------------------------------------|-------------------------------------|--------------------------------------|--------------------------------------|
| TC-06        | Foreign keys | Insert a child with a missing parent | Activity table row with a non-existent daily log | Insert rejected                     | [To be filled after final execution] | [To be filled after final execution] |
| TC-07        | Constraints  | Insert a row violating a constraint  | Row with a missing mandatory value               | Insert rejected                     | [To be filled after final execution] | [To be filled after final execution] |
| TC-08        | Joins        | Retrieve related rows                | Join of daily log with an activity table         | Related rows returned together      | [To be filled after final execution] | [To be filled after final execution] |
| TC-09        | Aggregations | Compute a summary                    | Aggregate query over a period                    | Correct totals or averages returned | [To be filled after final execution] | [To be filled after final execution] |
| TC-10        | Views        | Query a view                         | SELECT on the analytical view                    | Same result as the underlying query | [To be filled after final execution] | [To be filled after final execution] |

| Test Case ID | Module          | Scenario                                | Input                                 | Expected Result                       | Actual Result                        | Status                               |
|--------------|-----------------|-----------------------------------------|---------------------------------------|---------------------------------------|--------------------------------------|--------------------------------------|
| TC-11        | Triggers        | Fire the goal-progress trigger          | Insert or update of a relevant record | Goal progress updated automatically   | [To be filled after final execution] | [To be filled after final execution] |
| TC-12        | Transactions    | Roll back a failed multi-step operation | Second statement made to fail         | No partial change remains             | [To be filled after final execution] | [To be filled after final execution] |
| TC-13        | Cascading rules | Delete a parent with children           | Delete of a daily log with records    | Children handled per the defined rule | [To be filled after final execution] | [To be filled after final execution] |

### 8.4.3 Validation, negative and boundary testing

**Table 8.7:** Validation, negative and boundary test cases

| Test Case ID | Module         | Scenario                              | Input                                | Expected Result                         | Actual Result                        | Status                               |
|--------------|----------------|---------------------------------------|--------------------------------------|-----------------------------------------|--------------------------------------|--------------------------------------|
| TC-14        | All forms      | Submit an empty required field        | Empty value                          | Error shown; nothing stored             | [To be filled after final execution] | [To be filled after final execution] |
| TC-15        | Expenses       | Submit a value of the wrong type      | Text in a numeric field              | Request rejected with an error          | [To be filled after final execution] | [To be filled after final execution] |
| TC-16        | Authorisation  | Access another user's record          | Identifier owned by another user     | Access refused or record not returned   | [To be filled after final execution] | [To be filled after final execution] |
| TC-17        | Numeric fields | Value at the lower limit of its range | Smallest permitted value             | Accepted ([limits to be confirmed])     | [To be filled after final execution] | [To be filled after final execution] |
| TC-18        | Numeric fields | Value beyond the upper limit          | Value above the permitted maximum    | Rejected ([limits to be confirmed])     | [To be filled after final execution] | [To be filled after final execution] |
| TC-19        | Dates          | Duplicate daily log for the same day  | Second log for the same user and day | Behaviour per the final uniqueness rule | [To be filled after final execution] | [To be filled after final execution] |

The tests are a necessary check on a system whose purpose is to hold personal data reliably. They are not a performance study; no response-time or load figures are included, because none are available in the supplied materials.

## CHAPTER -9 DEPLOYMENT

### 9.1 Steps to Deploy

The deployment of LifeLog described here is a local deployment on a Windows or Linux machine, which corresponds to the technology stack and feasibility analysis of the project review. The steps follow the order in which the layers depend on each other: the database first, then the backend, and finally the dashboard.

**Step 1: install the prerequisites.** Install Java 17, Maven, MySQL 8 or later, Git and a modern web browser. **Step 2: obtain the source.** Clone the project repository from GitHub. **Step 3: prepare the database.** Start the MySQL server, create the database lifelog\_db and run the project SQL scripts to create the fourteen tables, views, procedures, triggers and indexes. **Step 4: configure the backend.** Set the datasource connection of the Spring Boot application to the MySQL server. **Step 5: build and start the backend.** Build with Maven and run the application. **Step 6: open the dashboard.** Open the dashboard in the browser, register or log in and begin recording data.

```
# Standard Maven commands for a Spring Boot project;
# confirm the exact commands and script names against the repository README
mvn clean package
mvn spring-boot:run
```

*[To be updated with actual implementation evidence] Names of the SQL scripts, the order in which they are run, the port used by the backend, and the way in which the dashboard files are served.*

### 9.2 Environment Configuration

The environment configuration consists of the settings that connect the layers: the address, port, user name and password of the MySQL server, the database name lifelog\_db, and the settings of the JPA/Hibernate layer. These are normally kept in the configuration files of the Spring Boot application. The credentials of the database must not be committed to the public repository; during academic development local configuration is acceptable, but a production-like deployment should read secrets from environment variables or a secrets store.

**Table 9.1:** Configuration items

| Item                   | Purpose                                              | Value                                               |
|------------------------|------------------------------------------------------|-----------------------------------------------------|
| Database name          | Schema used by the application                       | lifelog_db                                          |
| Database host and port | Location of the MySQL server                         | [To be updated with actual implementation evidence] |
| Database credentials   | Authentication to MySQL (keep out of source control) | Not recorded in this report                         |
| Backend port           | Port on which the Spring Boot application listens    | [To be updated with actual implementation evidence] |
| Java version           | Runtime for the backend                              | Java 17                                             |

### 9.3 Hosting of Frontend and Backend

At present the frontend and backend run locally. The dashboard consists of static files and can be served by the backend or by any simple web server, and it needs only the address of the backend REST API. No cloud hosting, container image or CI/CD pipeline is described in the project materials, so none is claimed.

A move to hosted deployment is described in Chapter 11 as future scope. It would involve three changes: placing the MySQL database on a managed service with automatic backups, running the Spring Boot application on a server reachable over the

internet, and protecting the traffic with HTTPS. Because the layers already communicate over HTTP and a configured database connection, the architecture would not change, which is the advantage of the layered design.

## CHAPTER -10 CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

The challenges described here are those that arise from the nature of the problem and the chosen design. They are presented as design challenges rather than as a dated record of defects, because the supplied materials do not include a development log.

#### 10.1.1 *Modelling many domains in one schema*

The first challenge was to model ten different domains in one coherent schema. Each domain has its own attributes, yet all must share the same user and the same notion of a day. A single wide table would have been easy to create but would have been full of empty values and repetition, while ten unconnected tables would have recreated the silos that LifeLog is meant to remove. Finding a structure that keeps domains separate and connected required identifying the backbone of users and daily\_logs.

#### 10.1.2 *Relationships and referential integrity*

With fourteen tables, correctness depends on many foreign-key relationships and on deciding what must happen when a parent row is removed. The choice between cascading deletion and refusing deletion has consequences for the user: cascading can remove more data than expected, while refusing can make removal awkward. Decisions of this kind had to be made for every parent–child pair and had to be consistent across the schema.

#### 10.1.3 *Mapping objects to relational tables*

Spring Data JPA and Hibernate map objects to rows, but a normalized relational schema with many relationships can be awkward to represent as objects. The mapping of relationships, the handling of keys, and the avoidance of unnecessary queries all need attention. Keeping the database design primary and mapping the application objects to it, rather than the reverse, was the approach that protected the quality of the schema.

#### 10.1.4 *Analytical queries across normalized tables*

A normalized schema moves complexity from storage to query. Questions that cross domains require multi-table joins and grouped aggregation, which are harder to write and to verify than queries on one table. Without care, the same complicated query would be repeated in several places.

#### 10.1.5 *Keeping goal progress current*

Goal progress depends on activity data that is recorded elsewhere. Relying on the user or on application code to update it after every change would be error-prone. Deciding where this logic should live was a design problem in its own right.

#### 10.1.6 *Connecting the layers*

Finally, the browser, the REST API, the persistence layer and the database have to agree on the shape of data and on how errors are reported. Faults can occur at any of the boundaries, and tracing one requires understanding all the layers.

### 10.2 Solutions Applied

**Table 10.1:** *Challenges and the solutions applied*

| Challenge                 | Solution applied                                                                         |
|---------------------------|------------------------------------------------------------------------------------------|
| Many domains in one model | A backbone of users and daily_logs with one table per activity domain, normalized to 3NF |

| Challenge                  | Solution applied                                                                                             |
|----------------------------|--------------------------------------------------------------------------------------------------------------|
| Referential integrity      | Primary and foreign keys, composite keys where required, referential integrity and cascading rules in InnoDB |
| Object–relational mapping  | Spring Data JPA / Hibernate between the REST API and MySQL                                                   |
| Complex analytical queries | SQL joins and aggregate queries packaged as views and stored procedures                                      |
| Current goal progress      | A trigger that updates goal progress automatically                                                           |
| Multi-step operations      | Transactions that commit or roll back as a unit                                                              |
| Layer integration          | A layered architecture with a defined request flow and a REST contract                                       |

The common principle behind these solutions is to put each responsibility in the place where it can be enforced most reliably. Rules about data go into the database; rules about requests go into the backend; rules about presentation go into the dashboard. The outcome is a system in which a mistake in one layer is not allowed to damage the integrity of the stored data.

### 10.3 Current Limitations or Known Bugs

The following limitations apply to the work reported here. They are stated openly so that the project is judged for what it is.

**Table 10.2:** *Current limitations*

| Area                 | Limitation                                                                                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Evidence             | Executed test results, measured performance figures and final screenshots are not contained in the materials used for this report and are shown as placeholders |
| Schema detail        | Column-level definitions, the exact composite keys and cascading actions must be recorded from the final DDL                                                    |
| Normalization        | The design is claimed to be in 3NF; Boyce–Codd Normal Form is not claimed                                                                                       |
| Deployment           | The system is deployed locally; there is no cloud hosting or backup service                                                                                     |
| Authentication       | Two-factor authentication is not implemented; the mechanism details are to be confirmed                                                                         |
| Clients              | Only a browser dashboard exists; there is no mobile application                                                                                                 |
| Data entry           | All data is entered manually; there are no integrations with fitness or banking services                                                                        |
| Reminders and export | There are no reminders and no PDF or CSV export                                                                                                                 |
| Scale                | The project is designed for personal use and no claim is made for large numbers of concurrent users                                                             |

No known software bugs are recorded in the supplied materials, and none are claimed or denied. A systematic execution of the test cases of Chapter 8 would be the proper way to list them.

## CHAPTER -11 FUTURE ENHANCEMENTS

Everything in this chapter is future scope. None of it is part of the implemented system described in the earlier chapters. The planned features are those identified in the project review.

### 11.1 Planned Features

#### ***11.1.1 Android and iOS companion application***

A mobile application would allow records to be entered at the moment they occur, which improves the completeness of the data. It would use the existing REST API, so the database and backend would be reused and only a new client would be needed.

#### ***11.1.2 Reminders for logs and goal deadlines***

Reminders would prompt the user to log the day and warn when a goal deadline is near. They would require stored deadlines or schedules and a mechanism that sends notifications, for which the existing goal data is the starting point.

#### ***11.1.3 Fitness and banking integrations***

Importing steps, workouts or bank transactions would reduce manual entry for the sleep, activity and expense domains. This needs careful treatment of authorisation, privacy and mapping of external data onto the existing tables.

#### ***11.1.4 Smarter habit recommendations***

Recommendations would use the patterns already revealed by joins and aggregation to suggest changes, for example in sleep or study routine. This would extend the analytical layer from reporting what happened to suggesting what might help.

#### ***11.1.5 Shared goals with friends or mentors***

Sharing would let a goal be visible to selected people. It changes the access model of the system, since data would no longer belong to one user alone, and would need explicit permissions in the schema.

#### ***11.1.6 PDF and CSV report export***

Export would let a user take a summary out of the system for study or discussion. It could be built on the existing views and queries by formatting their results as files.

#### ***11.1.7 Cloud-hosted database with backups***

A managed database would make the data available from any device and protect it against loss of the local machine. It would require configuring connection security and backup schedules.

#### ***11.1.8 Two-factor authentication***

A second authentication factor would strengthen the protection of sensitive personal data and would build on the existing user identity.

### 11.2 Possible Integrations or Optimisations

Beyond the planned features, a number of technical improvements would strengthen the system. They are suggestions for further work and are not claimed as part of the project.

At the database level, the index set could be reviewed against real query patterns once enough data has accumulated, using the query plan of MySQL to confirm whether an index helps. Views could be extended to cover additional analyses. At the application level, additional validation and a more complete error format would help the dashboard to explain rejections to

the user. At the infrastructure level, the move to a hosted database with scheduled backups would address data safety, and encrypted transport would protect the data in transit. Finally, predictive analytics and AI-assisted insights could be built on the cleaned, structured data that LifeLog already produces, which is a natural extension of the smarter habit recommendations listed above.

**Table 11.1:** *Future work and its dependence on the current design*

| Enhancement                | Builds on                                           |
|----------------------------|-----------------------------------------------------|
| Mobile application         | Existing REST API and database                      |
| Reminders                  | Goal data and a notification mechanism              |
| External integrations      | Existing activity tables as import targets          |
| Recommendations            | Joins, aggregates and views                         |
| Shared goals               | goals and users tables, with a new permission model |
| Export                     | Existing views and queries                          |
| Cloud database and backups | Existing MySQL schema                               |
| Two-factor authentication  | Existing user authentication                        |

## CHAPTER -12 CONCLUSION

### 12.1 Summary of the Project

LifeLog set out to answer a simple question: can the scattered records of everyday life be brought into one connected, queryable system? The project response is a normalized relational database named `lifelog_db` with fourteen related tables, built on the InnoDB engine and exposed through a Spring Boot REST backend and a browser dashboard. Personal records of ten domains are stored as structured, validated data owned by an authenticated user and connected through the backbone of users and daily logs.

The project shows how relational design handles a realistic problem. Reference tables avoid repetition, activity tables represent one-to-many relationships, goal progress is separated from goals and updated by a trigger, and the schema is organised so that no fact is stored twice. The database intelligence layer of joins, aggregates, views, stored procedures, triggers, transactions, referential integrity and indexing turns the stored data into information, and the dashboard presents that information as charts.

### 12.2 What Was Achieved

The achievements of the project are listed in the project review in terms of results and key achievements. Table 12.1 relates each objective of Section 1.4 to the evidence available for it. The evidence is qualitative and design-based, in line with the review, which states that no performance benchmarks are claimed.

**Table 12.1:** *Objectives achieved and the supporting evidence*

| Objective                            | Evidence from the project                                                                                                      |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Normalized relational database       | Fourteen related tables in <code>lifelog_db</code> , normalized up to 3NF, on InnoDB                                           |
| Reliable recording of activities     | Activity data centralized in one relational system; major activity domains stored as structured records                        |
| Tracking sleep, mood and screen time | Dedicated tables <code>sleep_logs</code> , <code>mood_logs</code> and <code>screen_time_logs</code> connected to the daily log |
| Goals and progress                   | <code>goals</code> and <code>goal_progress</code> tables, with trigger-updated progress                                        |
| Integrity                            | Primary, foreign and composite keys, referential integrity, cascading rules, constraints and triggers                          |
| Summaries                            | Joins and aggregation support cross-domain analysis; views and procedures give reusable operations                             |
| REST API                             | Spring Boot REST architecture gives a structured interface                                                                     |
| Dashboard                            | Dashboard-based visualization using Chart.js                                                                                   |

#### 12.2.1 Results discussion

The most significant result is the unified relational data model. Records that were previously held in separate places now share one schema and can be combined with a join, which makes cross-domain questions practical. The use of keys, constraints and referential integrity means that the data is trustworthy: invalid data and invalid relationships are prevented by the database itself. Triggers keep goal progress current without manual intervention, and transactions keep multi-step operations consistent. Views and procedures package the analytical logic so that it can be reused. On the application side, the REST architecture provides a clear interface and the dashboard makes the analytical results visible.

The limits of these results should be recognised. They describe the design and the functionality; recorded test outcomes and measurements will be added after final execution, as set out in Chapter 8.

### 12.2.2 Feasibility

**Table 12.2:** *Feasibility of the project*

| Dimension   | Assessment                                                                                          |
|-------------|-----------------------------------------------------------------------------------------------------|
| Technical   | The Java, Spring Boot and MySQL stack is mature, well documented and suited to a relational project |
| Economic    | All development technologies are open source, so there is no licensing cost                         |
| Operational | The browser-based dashboard needs no installation on the client side                                |
| Schedule    | The project fits within a semester project-based learning schedule                                  |

### 12.3 Skills Learned During Development

The project gave the team practical experience in database design from requirements to schema, including identification of entities and relationships, choice of keys, and normalization to 3NF. It developed skill in SQL, particularly joins, aggregation, views, stored procedures, triggers and transactions, and in understanding how referential integrity and cascading rules behave. On the application side it developed skill in building REST APIs with Spring Boot, mapping objects to tables with Spring Data JPA and Hibernate, and building a browser client with HTML, CSS, JavaScript, the Fetch API and Chart.js. The use of Git and GitHub, together with Maven, gave experience of the workflow of a team software project, and the work of designing, testing and documenting the system developed skills in planning and communication.

## CHAPTER -13 REFERENCES

### 13.1 Books

- [1] Silberschatz, Abraham; Korth, Henry F.; Sudarshan, S. *Database System Concepts*. McGraw-Hill.
- [2] Elmasri, Ramez; Navathe, Shamkant B. *Fundamentals of Database Systems*. Pearson.

### 13.2 Technology Documentation

- [3] MySQL Documentation — official reference for SQL, database administration, InnoDB and related database features.
- [4] Spring Boot Documentation — official reference for application configuration, web services and Spring Boot development.
- [5] Spring Data JPA Documentation — reference for repositories and persistence abstraction.
- [6] Hibernate ORM Documentation — reference for object-relational mapping.
- [7] Java Documentation — reference for Java 17 language and runtime features.

### 13.3 Tutorials and Learning Resources / APIs and Tools Used

- [8] MDN Web Docs — HTML, CSS, JavaScript and Fetch API reference.
- [9] Chart.js Documentation — chart configuration and visualization reference.
- [10] Git Documentation — source control reference.
- [11] GitHub Documentation — repository and collaboration reference.
- [12] Postman Documentation — API testing reference.

### 13.4 Documentation and Project Source Materials

- [13] LifeLog Review 1 presentation supplied by the project team.
- [14] LifeLog project documentation supplied for the project.
- [15] Sample project documentation supplied as the chapter-format reference.

References follow the list already present in the project documents. Edition numbers, years and URLs are to be added according to the citation style required by the department.

## CHAPTER -14 APPENDICES

### 14.1 Appendix A — Screenshots of the Application

The screenshots below are to be taken from the running system. Each placeholder carries the caption under which the real image should be inserted so that figure numbering stays consistent.

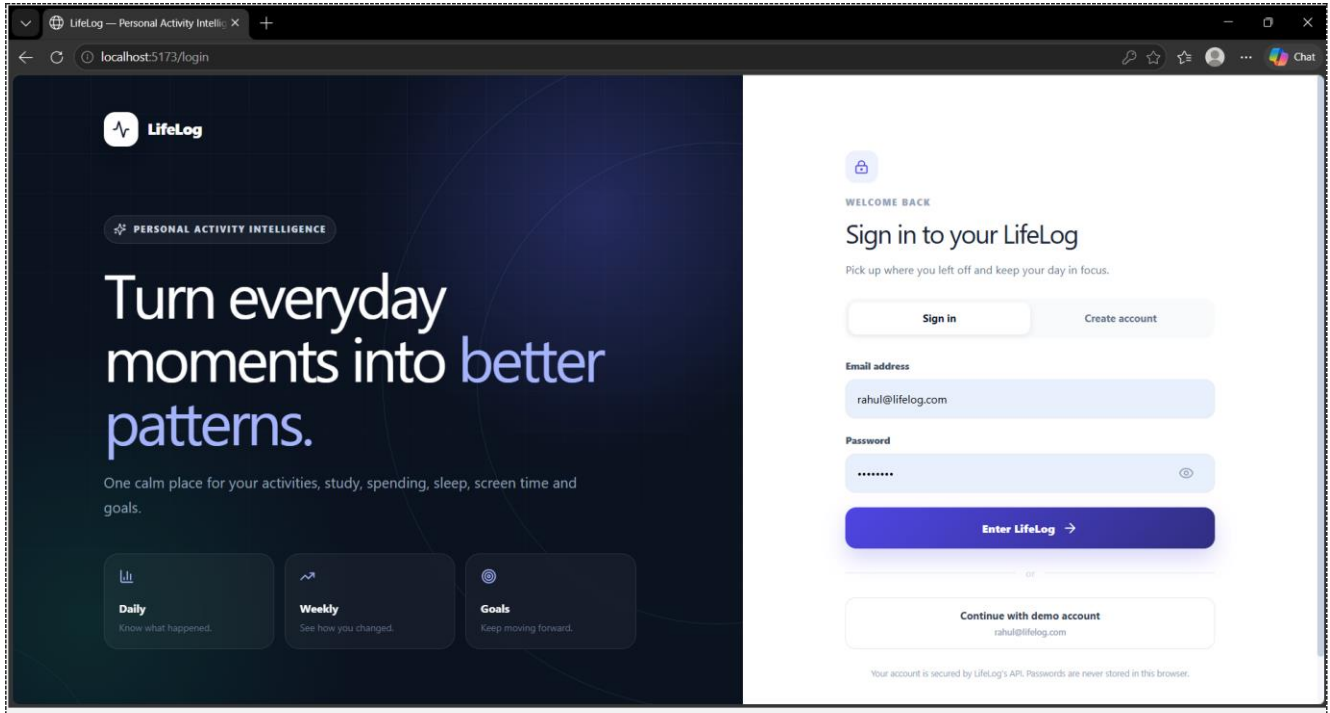


Figure 14.1: Login / registration screen

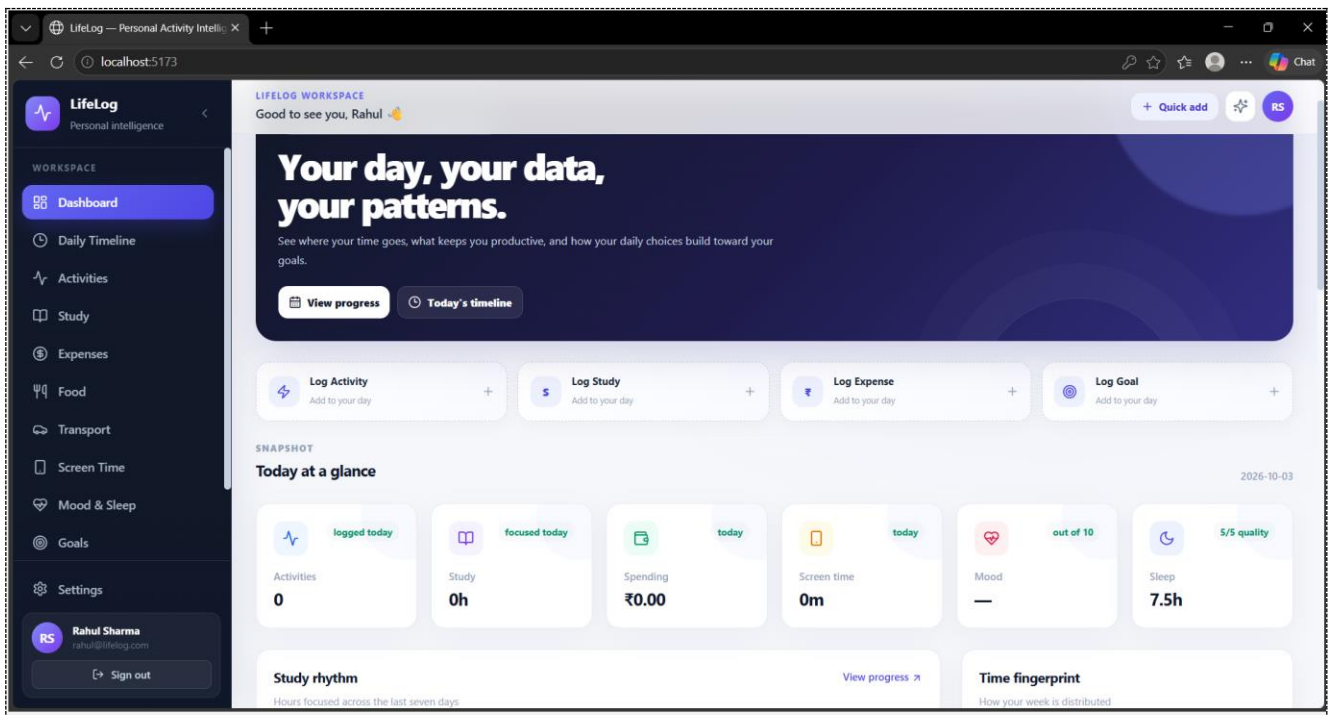


Figure 14.2: Daily log and activities screen

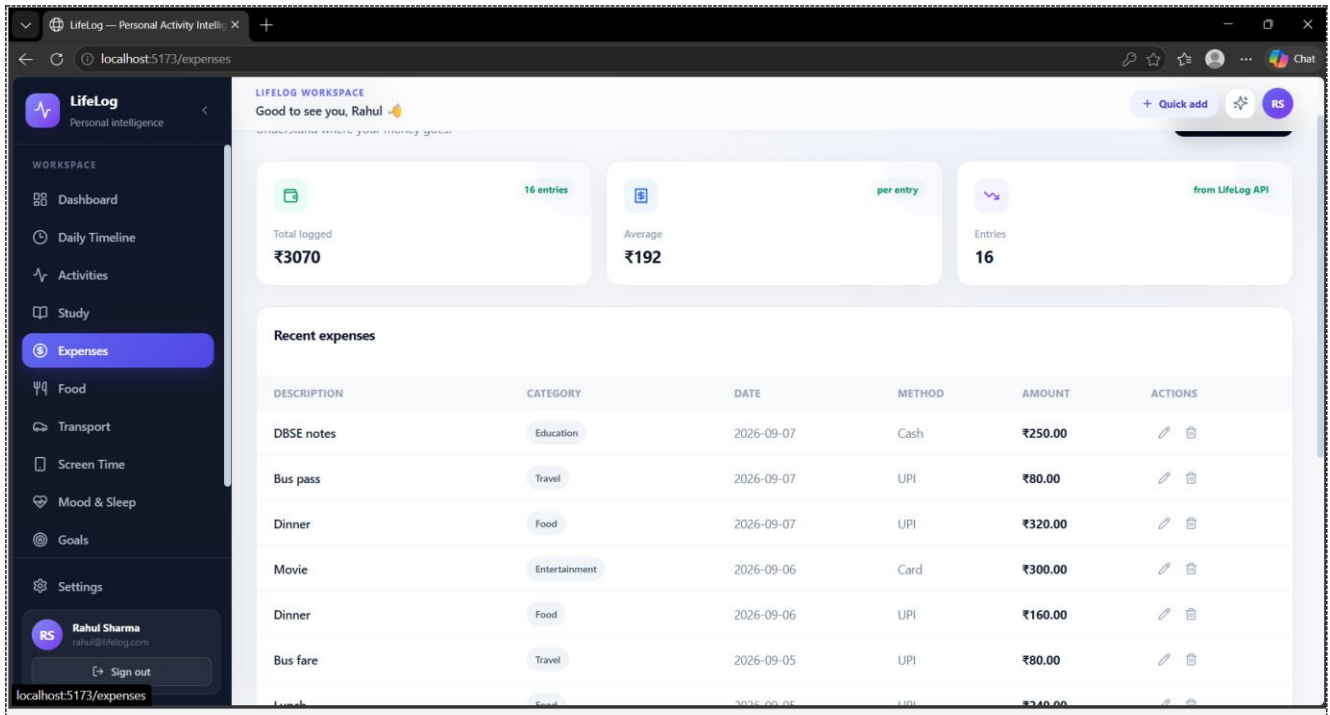


Figure 14.3: Expenses screen

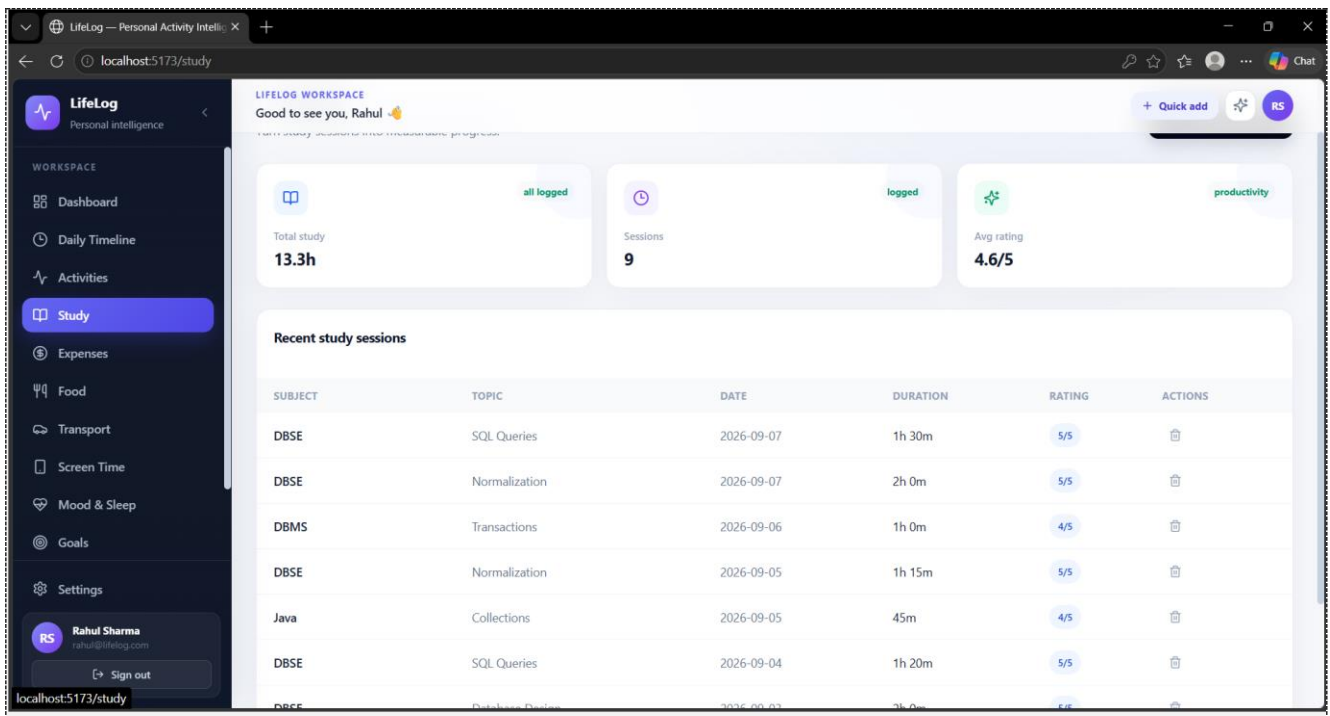


Figure 14.4: Study, health and transport screens

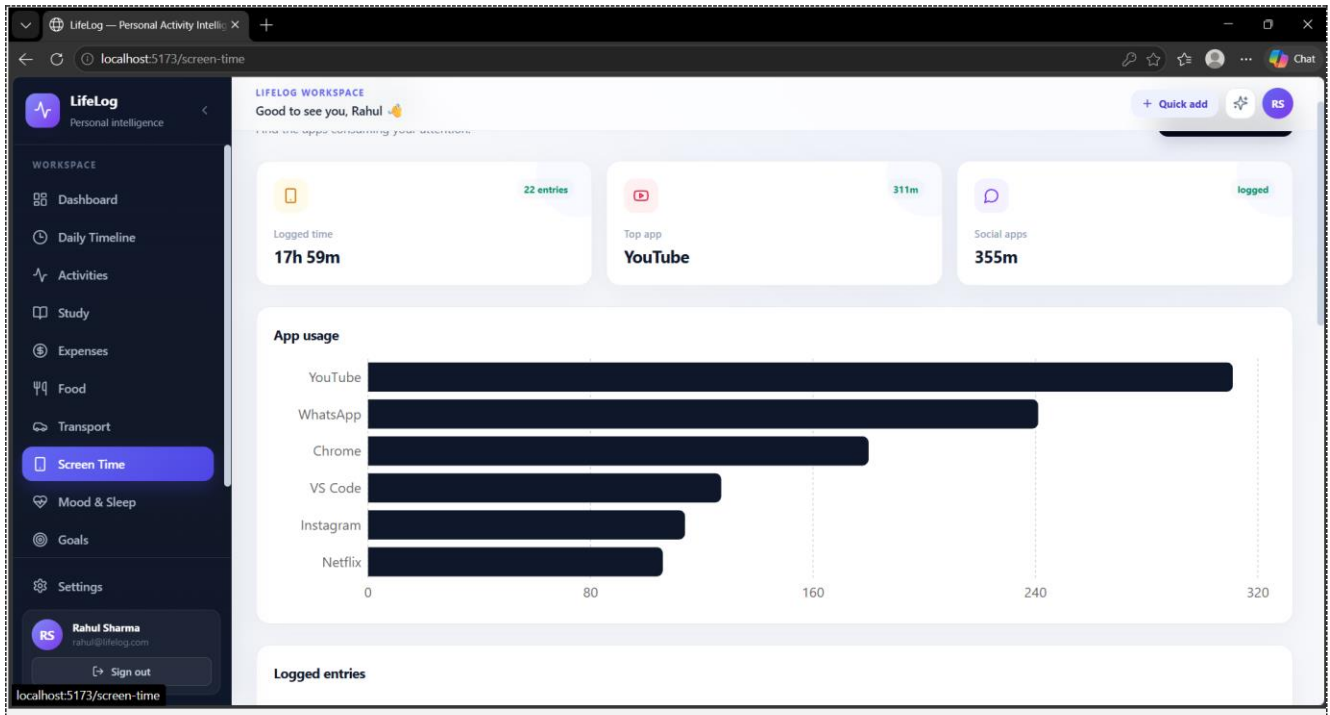


Figure 14.5: Screen time and goals screens

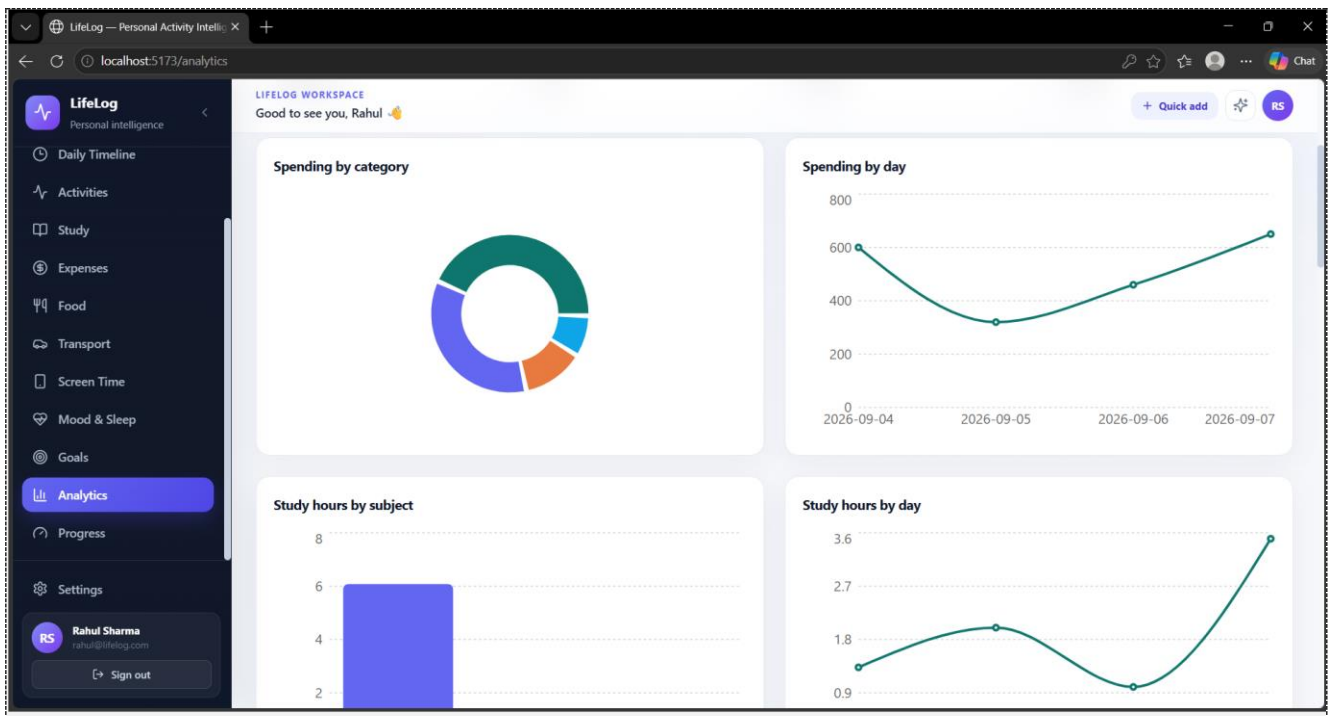


Figure 14.6: Analytics dashboard with Chart.js charts

## 14.2 Appendix B — Architecture Diagram

The architecture diagram of the project review is reproduced in Figure 4.1 and explained in Chapter 4.

## 14.3 Appendix C — ER Diagram

The simplified ER diagram of the project review is reproduced in Figure 5.1. The detailed ER diagram with attributes should be inserted below.

## 14.4 Appendix D — Database Schema

The schema consists of the fourteen tables listed in Table 14.1. The column definitions, data types, keys, constraints and cascading actions must be copied from the final DDL.

**Table 14.1:** *Schema listing*

| Table               | Parent table(s)                            | Columns, keys, constraints, cascading action        |
|---------------------|--------------------------------------------|-----------------------------------------------------|
| users               | —                                          | [To be updated with actual implementation evidence] |
| daily_logs          | users                                      | [To be updated with actual implementation evidence] |
| activity_categories | —                                          | [To be updated with actual implementation evidence] |
| locations           | —                                          | [To be updated with actual implementation evidence] |
| activities          | daily_logs, activity_categories, locations | [To be updated with actual implementation evidence] |
| expenses            | daily_logs                                 | [To be updated with actual implementation evidence] |
| food_logs           | daily_logs                                 | [To be updated with actual implementation evidence] |
| study_sessions      | daily_logs                                 | [To be updated with actual implementation evidence] |
| transport_logs      | daily_logs                                 | [To be updated with actual implementation evidence] |
| screen_time_logs    | daily_logs                                 | [To be updated with actual implementation evidence] |
| mood_logs           | daily_logs                                 | [To be updated with actual implementation evidence] |
| sleep_logs          | daily_logs                                 | [To be updated with actual implementation evidence] |
| goals               | users                                      | [To be updated with actual implementation evidence] |
| goal_progress       | goals                                      | [To be updated with actual implementation evidence] |

Database objects (views, stored procedures, triggers, indexes): [To be updated with actual implementation evidence]

## 14.5 Appendix E — Sample Code Snippets

Representative controller, service, repository, entity and SQL snippets are to be inserted here from the final source code. [To be updated with actual implementation evidence]

## 14.6 Appendix F — Installation / Setup Instructions

The installation procedure is described in Section 9.1. It consists of installing Java 17, Maven and MySQL 8 or later, cloning the repository, creating lifelog\_db with the project scripts, configuring the datasource, building and starting the Spring Boot application, and opening the dashboard in a browser.

## 14.7 Appendix G — User Manual / Guide

**Registering and logging in.** Create an account and log in so that your records are stored under your own identity. **Starting a day.** Open the dashboard and create or select the log for the day. **Recording data.** Choose the form of the domain, such as expenses, study, food, transport, screen time, mood or sleep, enter the values and save. **Locations and categories.** Add the places and categories that you use often so that you can select them later. **Goals.** Create a goal; its progress is updated as you record data. **Reviewing.** Use the dashboard to view records and charts, and correct or remove entries if needed.

## 14.8 Appendix H — API Documentation

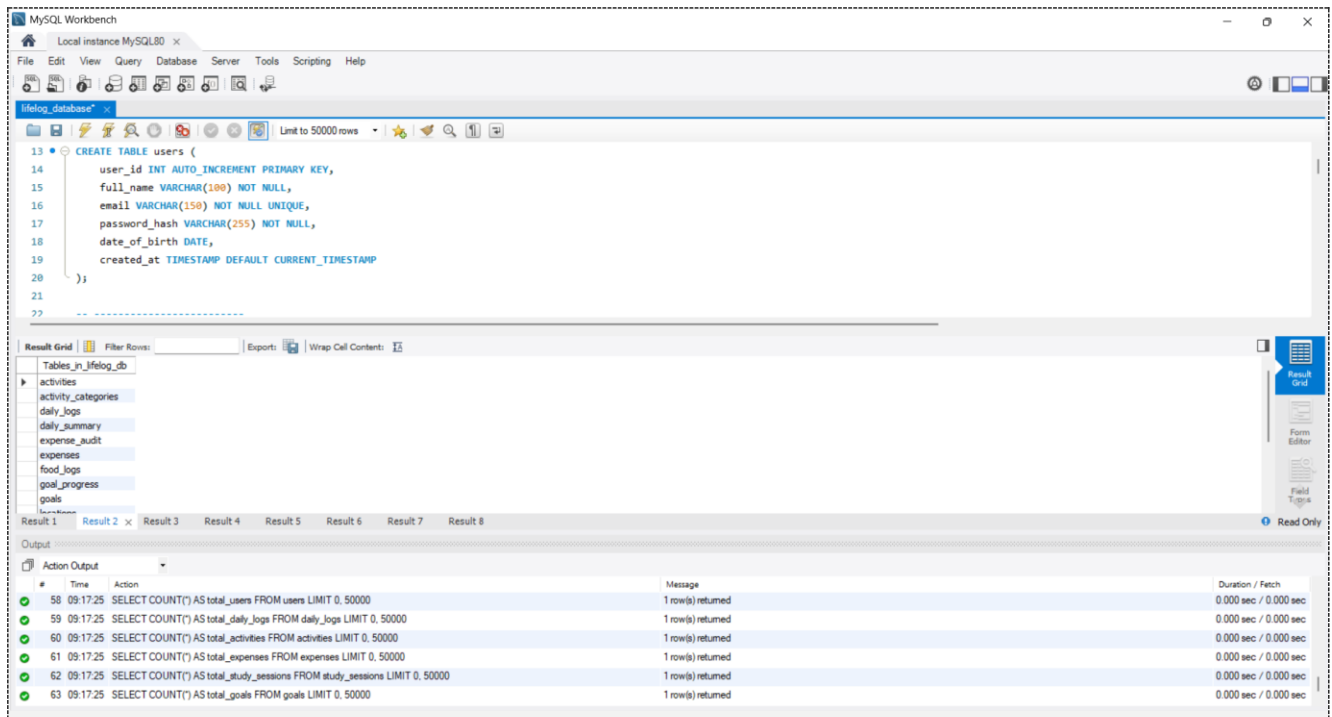
The list of REST endpoints is to be filled in from the Spring Boot source code.

**Table 14.2: REST endpoint listing**

| Module             | HTTP method                                         | Endpoint                                            | Purpose                                             | Request / response                                  |
|--------------------|-----------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------|
| Daily activities   | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Study sessions     | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Expenses           | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Food and nutrition | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Transport          | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Screen time        | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Mood               | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Sleep              | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Locations          | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |
| Goals and progress | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] | [To be updated with actual implementation evidence] |

## 14.9 Appendix I — Testing Evidence

Screenshots of Postman requests, query outputs and trigger, view, procedure and transaction demonstrations are to be inserted here after the final execution. [To be filled after final execution]



**Figure 14.8:** Query output, trigger, view, procedure and transaction evidence

## 14.10 Appendix J — Project Repository

GitHub repository URL: [To be updated with actual implementation evidence]

## 14.11 Geo-Tag Photos with Guide

[To be updated with actual implementation evidence]

## 14.12 Review Forms with Guide Signatures

[To be updated with actual implementation evidence]